



Qualcomm Linux 摄像头指南

80-70022-17SC AA

October 28, 2025

Contents

1	摄像头文档	3
1.1	摄像头概述	3
1.2	摄像头流传输	3
1.3	增强摄像头输出	3
1.4	摄像头故障排查	4
2	摄像头概述	5
2.1	摄像头组件	5
2.2	前提条件	7
3	摄像头流传输	8
3.1	设置摄像头	8
3.2	选择流API	23
3.3	使用GStreamer API 进行摄像头流传输	23
3.4	USB 摄像头	45
3.5	网络摄像头	45
4	增强摄像头输出	47
4.1	支持使用离线IFE 的多摄像头	47
4.2	支持软件TNR/MCTF	49
4.3	启用高动态范围	50
4.4	启用EIS 和LDC	53
4.5	高级功能并发	59
4.6	启用多个ROI 流	63
4.7	自动切换Linear 与SHDR 模式	65
5	摄像头故障排查	67

1 摄像头文档

连接摄像头传感器，启用摄像头用例，并了解可用的高级功能。

1.1 摄像头概述

[摄像头组件](#)

查看摄像头子系统的组件。

[前提条件](#)

设置您的设备以实现基本摄像头功能。

1.2 摄像头流传输

[设置摄像头](#)

将摄像头传感器连接到您的参考硬件平台。

[选择流API](#)

查看可用的流API 选项。

[使用GStreamer API 进行流传输](#)

使用GStreamer API 进行摄像头流传输。

[使用V4L2 API 进行流传输](#)

使用V4L2 框架进行摄像头流传输。

1.3 增强摄像头输出

[启用高动态范围](#)

启用交错式高动态范围(SHDR) 功能。

[启用EIS 和LDC](#)

启用电子图像稳定(EIS) 和镜头失真校正(LDC) 功能。

[启用除雾](#)

启用除雾功能。

1.4 摄像头故障排查

[传感器流传输故障排查](#)

摄像头问题故障排查。

2 摄像头概述

本文档介绍了通过MIPI CSI 接口接收数据的摄像头子系统。

USB 摄像头数据通过USB 接口传输，摄像头子系统不涉及USB 摄像头数据传输。请参阅[USB 摄像头配置](#)了解USB 摄像头的使用。

网络摄像头数据通过网络接口传输，摄像头子系统不涉及网络摄像头数据传输。设备可以通过GStreamer [rtspsrc](#) 插件接收网络摄像头数据。

2.1 摄像头组件

下图显示了Qualcomm 摄像头的组件。



以下组件由Qualcomm 提供:

组件	说明
GST 摄像头插件(qtiqmmfsrc)	Qualcomm 摄像头子系统的GStreamer 插件
摄像头核心软件	Qualcomm 专有的摄像头软件，提供开发摄像头传感器驱动程序、摄像头调整和自定义软件节点的接口
摄像头核心驱动程序	下游Linux 内核中的Qualcomm 摄像头子系统驱动程序
CamSS V4L2 驱动程序	上游Linux 内核中的Qualcomm 摄像头子系统驱动程序

Qualcomm 提供了一个GStreamer 插件，使应用程序开发者能够与Qualcomm 摄像头子系统进行交互。有关应用程序开发，请参阅[GStreamer-camera-application](#)。

Qualcomm 为摄像头传感器驱动程序开发者和IQ 调试工程师提供了一个接口，用于开发自己的传感器驱动程序并执行定制IQ 调试。请参见[Camera Addendum document](#) 中的Camera Sensor Driver Development and Tuning。

要开发摄像头传感器驱动程序并定制IQ 调试，需要访问meta-qcom-extras 层。有关访问级别，请参见[访问级别映射](#)。

2.2 前提条件

- 按照[Qualcomm Linux 编译指南](#)中所述搭建基础架构
- 刷新最新的软件版本，将其写入至开发板中
- 设置SSH 连接：
 1. 通过执行[使用SSH](#) 中提到的步骤，在Permissive 模式下启用SSH。
 2. 要连接到设备，可运行以下命令：

```
ssh root@<device_IP_address>
```

例如，如果设备的IP 地址为10.92.160.222，执行以下命令：

```
ssh root@10.92.160.222
```

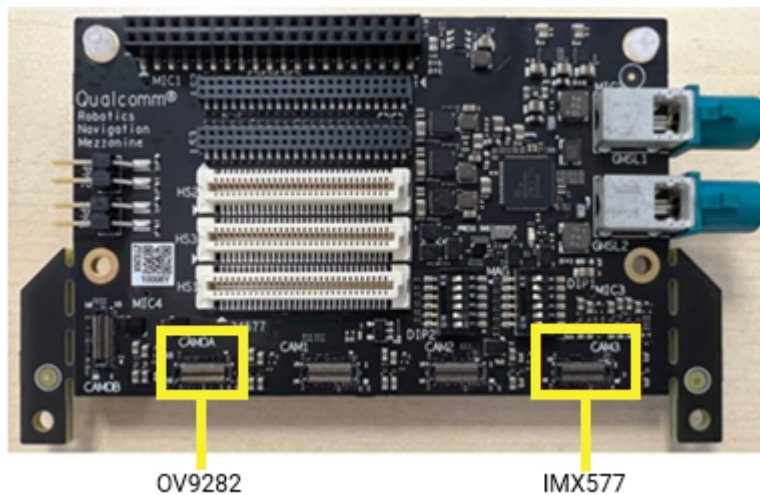
3 摄像头流传输

本页面介绍如何将摄像头传感器连接到参考硬件平台，并提供有关可用API 的信息。

3.1 设置摄像头

QCS6490

机器视觉**Mezzanine** 板卡上的**RB3G2 MIPI CSI** 连接 OV9282 和IMX577 是默认的摄像头传感器。将OV9282 模块连接到CAM0A，将IMX577 模块连接到CAM3 端口，如下所



示:

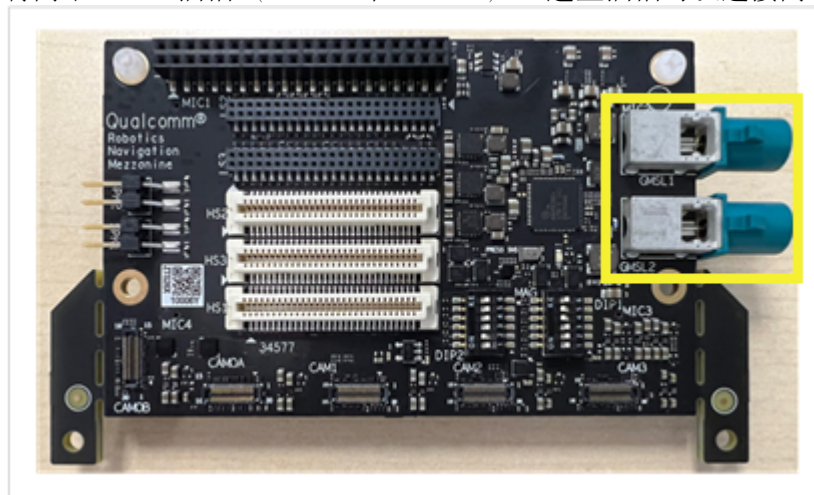


转接板上的**SW2300**

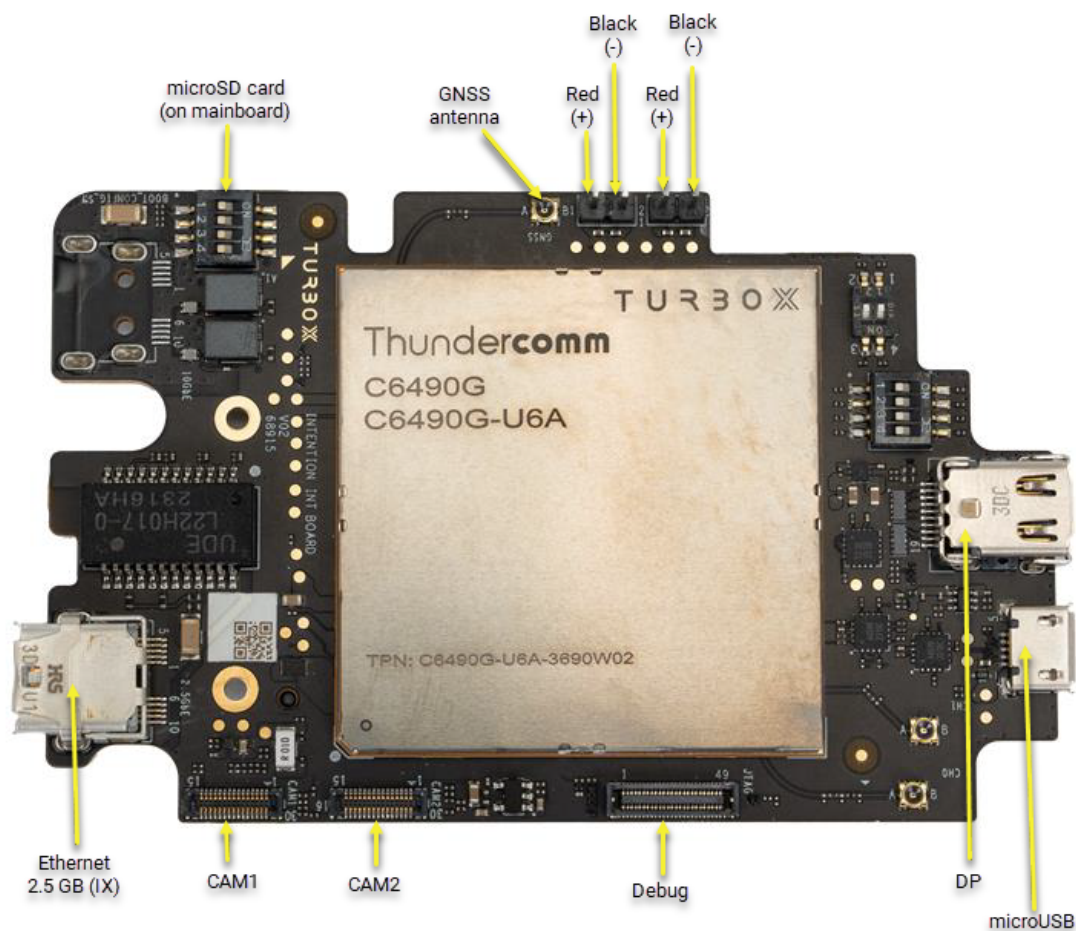
DIP 开关 要使用机器视觉Mezzanine 板卡的CAM3 插槽，必须将转接板上SW2300 DIP 开关1 设置为OFF。默认情况下，此开关设置为OFF。要使用转接板的CAM1 和CAM2 插



槽：机器视觉Mezzanine 板上的RB2G2 GMSL 连接 Qualcomm RB3G2 机器视觉套件平台的Mezzanine 板卡有两个GMSL 插槽（GMSL1 和GMSL2）。这些插槽可以连接两

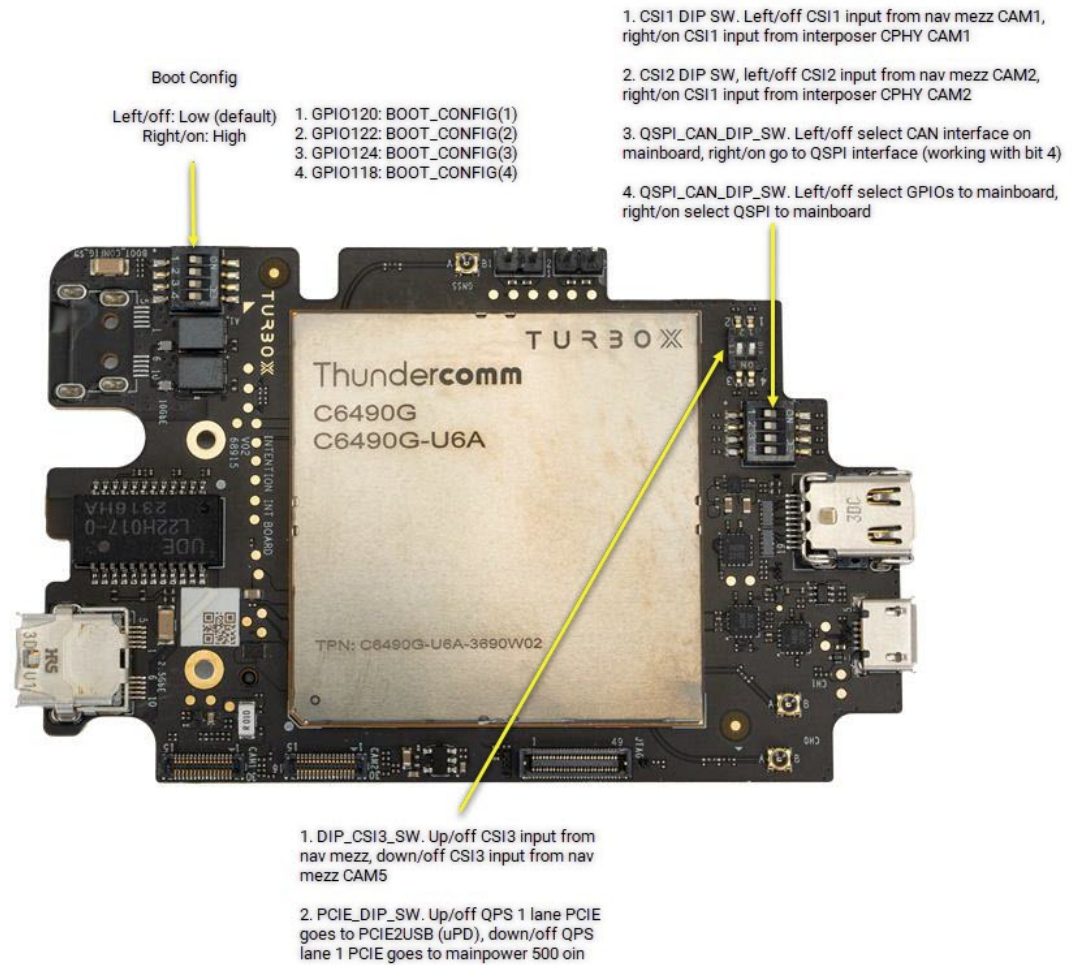


个GMSL 摄像头。
GMSL1 插槽无需任何硬件设置即可使用。要使用GMSL2 插槽，必须将转接板上的SW2300 DIP 开关1 设置为ON。默认情况下，此开关设置为OFF。请参阅[转接板上的SW2300 DIP 开关](#)。转接板上的RB3G2 MIPI CSI 摄像头 将OV9282 模块连接至CAM1，将IMX577 模块连接至转接板上



的CAM2。

将CSI1_DIP_SW 和CSI2_DIP_SW 设置为ON（如右侧图片所示）。如下图#7 所示，开关默认设置为OFF。将其更改为ON，以使用核心套件上



的CAM/CAM2。

支持的分辨率和功能 下表显示了每个摄像头模块支持的分辨率。

分辨率	宽高比	IMX577 (CAM3)	OV9282 (CAM0A)	AR0231 (GMSL)
4000 x 3000	4:3	是	否	否
3840 x 2160	16:9	是	否	否
2976 x 2976	1:1	是	否	否
2592 x 1940	4:3	是	否	否
2048 x 1536	4:3	是	否	否
1920 x 1440	4:3	是	否	否
1928 x 1208	16:10	是	否	是
1920 x 1080	16:9	是	否	是
1440 x 1080	4:3	是	否	是
1280 x 720	16:9	是	是	是
1024 x 768	4:3	是	是	是
800 x 600	4:3	是	是	是

分辨率	宽高比	IMX577 (CAM3)	OV9282 (CAM0A)	AR0231 (GMSL)
640 x 480	4:3	是	是	是
640 x 360	16:9	是	是	是
320 x 240	4:3	是	是	是

下表显示了每个摄像头模块支持的功能。

功能	IMX577 (CAM3)	OV9282 (CAM0A)	AR0231 (GMSL)
SHDR	是	否	否
LDC	是	否	否
EIS	是	否	否

QCS9075

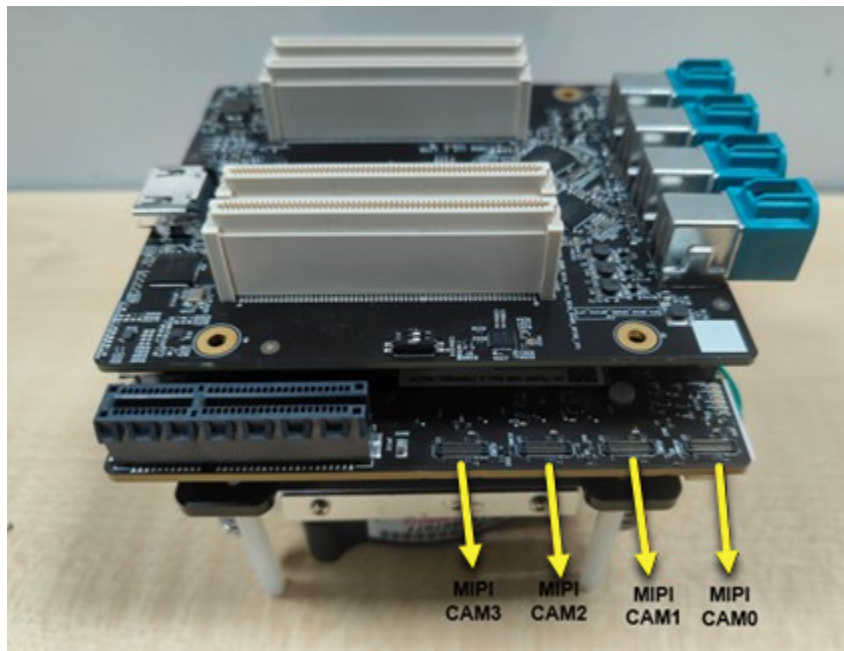
本节介绍如何在QCS9075 参考硬件平台RB8 设备上连接摄像头传感器。

在**RB8** 上连接**MIPI CSI** 摄像头

RB8 将提供一个称为核心套件的套件版本。此核心套件仅支持MIPI CSI 摄像头。使用需要连接到核心套件的单独附加GMSL mezzanine 板卡来支持GMSL 摄像头。

在RB8 EVT（核心套件+ GMSL mezzanine）硬件中，RB8 核心套件/主板上四个MIPI CSI（C/D-PHY）接口，GMSL mezzanine 板卡上有四个GMSL 端口（0 到3）。

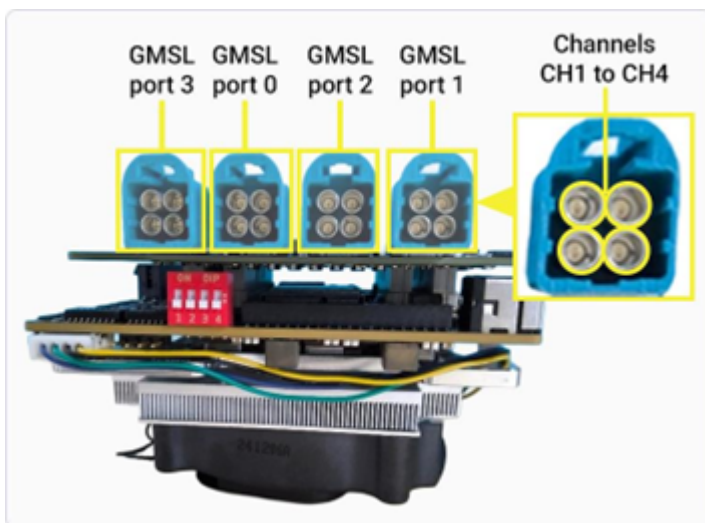
下图显示了RB8 核心套件/主板上的MIPI 接口。



连接**RB8** 上的**GMSL** 摄像头

GMSL mezzanine 板卡上有四个GMSL 端口（0 到3）。每个GMSL 端口都与一个MAX96724 四路GMSL 解串器连接。每个解串器都连接到一个CSI。

下图显示了RB8 设备上的GMSL 端口。请注意，GMSL 端口编号不是按顺序排列的- 它根据GMSL 端口连接到的CSI 索引定义。



Note: 在当前版本中，启用三个GMSL 端口（0、2 和3）来连接GMSL 摄像头。端口1 未启用。

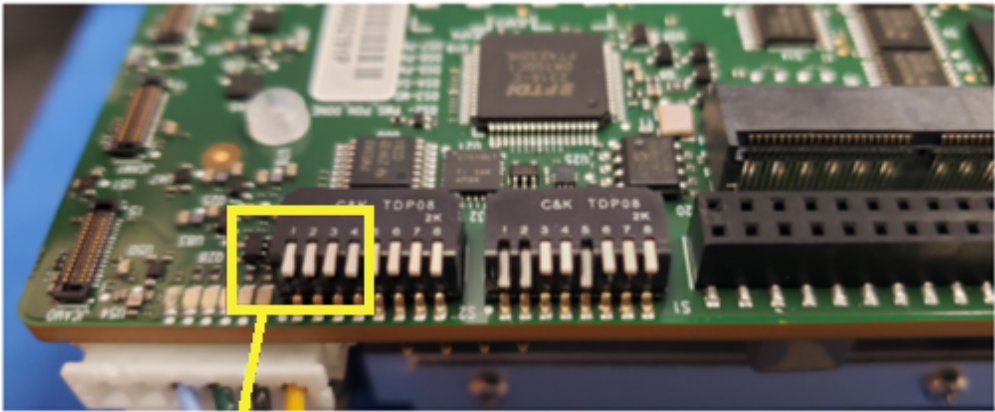
- 每个GMSL 端口包含四个插槽（1 至4，也称为通道），并支持将四个GMSL 摄像头连接到单个端口。
- 当前版本支持OX03f10 Bayer GMSL 和OX03f10 YUV GMSL 摄像头。
- 端口0 支持OX03f10 Bayer GMSL 摄像头。端口2 和端口3 支持OX03f10 YUV GMSL 摄像头。因此，您可以在GMSL 端口0 上连接四个OX03F10 Bayer GMSL 摄像头，在GMSL 端口2 上连接四个OX03F10 YUV GMSL 摄像头，在GMSL 端口3 上连接四个OX03F10 YUV GMSL 摄像头。

Note: 在当前版本中，连接到端口0 的OX03f10 Bayer GMSL 摄像头将无法在RB8 硬件上运行。此问题将在下一个版本中修复。

设置DIP 开关

QCS9075 有四个CSI PHY。每个CSI PHY 都通过核心套件/主板上的DIP 开关路由到MIPI 摄像头端口或GMSL 摄像头端口。EVT 和DVT/PVT 硬件版本的开关不同：

- RB8 EVT 硬件的DIP 开关设置：下图显示了核心套件/主板上的DIP 开关。



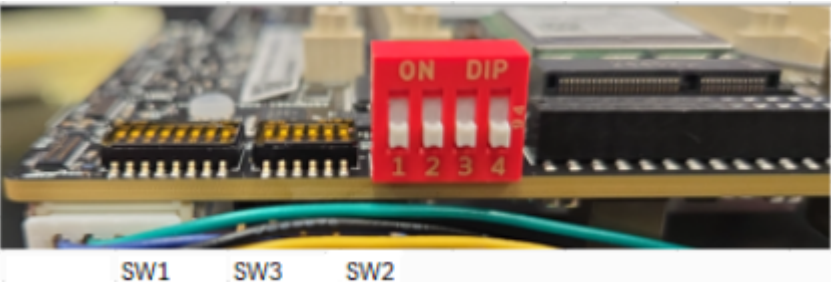
Switch 2 Switch 1

Pins 1, 2, 3, and 4 of SW2 should be moved up to enable MIPI and moved down to enable GMSL sensors

下表说明了使用MIPI 和GMSL 摄像头端口所需的DIP 开关SW2 设置。

开关	OFF（出厂默认）	ON	开启时连接	关闭时连接（出厂默认）
SW2 - 1	高	低	CSI0 连接至GMSL mezzanine	CSI0 连接至主板
SW2 - 2	高	低	CSI1 连接至GMSL mezzanine	CSI1 连接至主板
SW2 - 3	高	低	CSI2 连接至GMSL mezzanine	CSI2 连接至主板
SW2 - 4	高	低	CSI3 连接至GMSL mezzanine	CSI3 连接至主板

- RB8 DVT/PVT 硬件的DIP 开关设置： 下图显示了核心套件/主板上的DIP 开关。



下表说明了使用MIPI 和GMSL 摄像头端口所需的DIP 开关SW1 设置。DIP 开关上写着“ON”，表示启用的开关方向。

开关	开启时连接	关闭时连接（出厂默认）
SW1 - 5	CSI0 连接至GMSL mezzanine	CSI0 连接至主板

SW1 - 6	CSI1 连接至GMSL mezzanine	CSI1 连接至主板
SW1 - 7	CSI2 连接至GMSL mezzanine	CSI2 连接至主板
SW1 - 8	CSI3 连接至GMSL mezzanine	CSI3 连接至主板

支持的分辨率和功能

下表显示了RB8 平台上每个摄像头模块支持的分辨率。

分辨率	宽高比	0X3F10 Bayer GMSL	0X3F10 YUV GMSL	OV9282 (CAM0A)
1920 x 1536	5:4	否	是	否
1920 x 1440	4:3	否	是	否
1928 x 1208	16:10	否	是	否
1920 x 1080	16:9	否	是	否
1824 x 1536	19:16	是	是	否
1440 x 1080	4:3	是	是	否
1280 x 720	16:9	是	是	是
1024 x 768	4:3	是	是	是
800 x 600	4:3	是	是	是
640 x 480	4:3	是	是	是
640 x 360	16:9	是	是	是
320 x 240	4:3	是	是	是

QCS9075 不支持SHDR、LDC 和EIS 等高级功能。

RB8 支持并发摄像头

下表说明了当前版本中支持的并发摄像头用例。

- **RB8 MIPI 传感器：**该软件支持OV9282 MIPI 摄像头。下表描述了RB8 平台对OV9282 MIPI 摄像头的支持。

MIPI 摄像头	支持的分辨率和FPS	CAM0	CAM1	CAM2	CAM3	备注
OV9282	1280x720, 30 FPS	支持	支持	支持	支持	任何摄像头都可以独立运行。 最多两个摄像头可以同时运行并进行实时处理。所有四个摄像头可以同时运行并使用离线功能。

MIPI 摄像头	支持的分辨率和FPS	CAM0	CAM1	CAM2	CAM3	备注
----------	------------	------	------	------	------	----

- RB8 GMSL 传感器：软件支持OX03F10 Bayer 和OX03F10 YUV GMSL 摄像头。下表描述了RB8 平台对GMSL 摄像头的支持。

GMSL 端口	支持的分辨率和FPS	GMSL 传感器	注意
端口0	1824x1536, 30 FPS	3F10 GMSL Bayer	GMSL 端口0 上可以连接四个OX03F10 BayerGMSL 摄像头。 任何摄像头都可以独立运行。所有四个摄像头可同时运行。 在当前版本中，连接到端口0的OX03f10 Bayer GMSL 摄像头无法在RB8 硬件上运行。因此，无法验证该端口。此问题将在下一个版本中修复。

GMSL 端口	支持的分辨率和FPS	GMSL 传感器	注意
端口2	1920x1080, 30 FPS	3F10 GMSL YUV	GMSL 端口2 上可以连接四个OX03F10 YUV GMSL 摄像头。任何摄像头都可以独立运行。所有四个摄像头可以同时运行。
端口3	1920x1080,30 FPS	3F10 GMSL YUV	GMSL 端口2 上可以连接四个OX03F10 YUV GMSL 摄像头。任何摄像头都可以独立运行。所有四个摄像头可以同时运行。
端口2 + 端口3 的GMSL 并发	1920x1080,30 FPS	3F10 GMSL YUV	可连接八个OX03F10 YUV GMSL 摄像头-端口2 上四个，端口3 上四个。任何摄像头都可以独立运行。所有八个摄像头可以同时运行。

- RB8 GMSL 传感器+ MIPI 传感器并发：RB8 平台支持以下GMSL 传感器+ MIPI 传感器并发。

	CAM0	CAM1	GMSL 端口2
摄像头传感器和支持的分辨率	OV9282: 1280x720, 30 FPS	OV9282: 1280x720, 30 FPS	OX03F10 YUV G

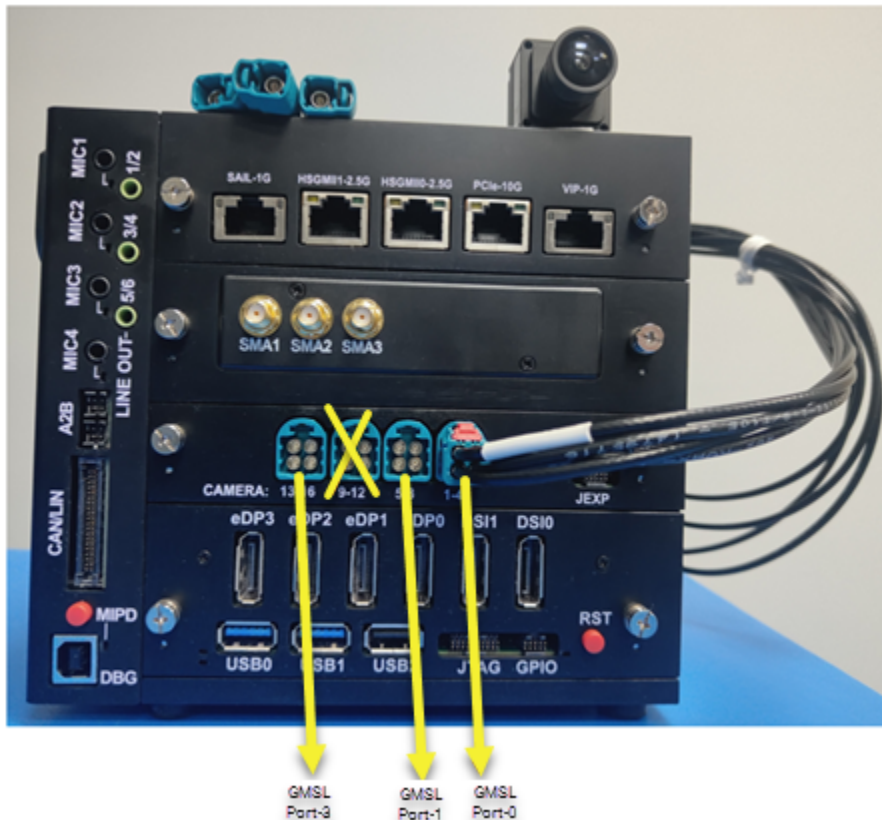
QCS8275

本节介绍如何在QCS8275 参考平台Ride SX 硬件上连接摄像头模块。

在Ride SX 硬件上连接GMSL 摄像头模块

- 在QCS8275 参考平台Ride SX 硬件中，存在四个GMSL 端口（0 至3）。支持三个GMSL 端口（0、1 和3）连接GMSL 摄像头。端口2 是虚拟端口。每个GMSL 端口都与一个MAX96724 四路GMSL 解串器连接。
- 此硬件上没有可用的MIPI CSI 摄像头端口。
- 每个GMSL 端口包含四个插槽（1 至4，也称为通道），并支持将四个GMSL 摄像头连接到单个端口。
- 使用此版本中的现有软件配置，单个GMSL 端口仅支持在其四个插槽中的任何一个上连接一个GMSL 摄像头。GMSL 端口0 支持单个Bayer 摄像头，端口1 和端口3 各支持单个YUV 摄像头。这些端口上的任何插槽都可以用来连接摄像头。
- 当前版本支持OX03f10 Bayer GMSL 和OX03f10 YUV GMSL 摄像头。您可以在GMSL 端口0 上连接一个OX03F10 Bayer GMSL，在端口1 和端口3 上分别连接两个OX03F10 YUV GMSL 摄像头。

下图显示了OX03F10 Bayer GMSL 摄像头在GMSL 端口0 上的连接。



支持的分辨率和功能

下表显示了Ride SX 平台上每个摄像头模块支持的分辨率。

分辨率	宽高比	0X3F10 Bayer GMSL	0X3F10 YUV GMSL
1920 x 1536	5:4	否	是
1920 x 1440	4:3	否	是
1928 x 1208	16:10	否	是
1920 x 1080	16:9	否	是
1824 x 1536	19:16	是	是
1440 x 1080	4:3	是	是
1280 x 720	16:9	是	是
1024 x 768	4:3	是	是
800 x 600	4:3	是	是
640 x 480	4:3	是	是
640 x 360	16:9	是	是
320 x 240	4:3	是	是

QCS8275 不支持SHDR、LDC 和EIS 等高级功能。

3.2 选择流API

Qualcomm Linux 支持以下摄像头API。

- **GStreamer API**

GStreamer 是一个开源多媒体框架。Qualcomm 提供了GStreamer 插件（qtiqmmfsrc），允许开发人员控制应用程序中的摄像头子系统。有关详细信息，请参阅[Qualcomm GStreamer 插件](#)。

- **V4L2 API**（仅限QCS6490）

V4L2 是Linux 内核中的一个框架，提供对视频采集、视频输出和其他多媒体设备的支持。开发者可以使用V4L2 API 来操作摄像头。V4L2 API 使用CAMSS 驱动程序，适合只需要从摄像头获取原始图像的开发者。

3.3 使用GStreamer API 进行摄像头流传输

gst-launch-1.0 是一个命令行GStreamer 实用工具，用于构建和运行GStreamer pipeline。Pipeline 被指定为元素的集合，其属性以感叹号 (!) 分隔。

前提条件

要使用[gst-launch-1.0](#)和GStreamer 插件，必须在设备上安装QIM SDK (meta-qcom-qim-product-sdk)。有关QIM SDK 的编译和安装信息，请参阅[Qualcomm Linux 编译指南](#)。

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

在SSH 终端中运行以下命令：

```
# mount -o rw,remount /usr
```

QCS6490

Note: 确保MIPI 摄像头已连接到CSI 插槽。OV9282 MIPI 摄像头应连接到CAM 0A 插槽，而IMX577 MPI 摄像头应连接到CAM3 插槽。

单摄像头流启动

1. 在设备终端运行以下命令：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! 'video/x-raw,format=NV12,\width=1280,height=720,framerate=30/1' ! fakesink
```

2. 此命令在30 FPS 配置下以720p 启动摄像头。来自摄像头传感器的帧被fakesink 丢弃。如果gst pipeline 状态更改为“PLAYING”，如下所示，则表示摄像头正在运行。由于此命令会将摄像头帧dump 到fakesink，因此不会在设备上保存任何内容。

```
gbm_create_device(187): Info: backend name is: msm_drm
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
```

若要停止摄像头，请按CTRL+C。

视频编码

1. 在设备终端运行以下命令：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
```



```
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
mux_avc.mp4
```

此命令以720p、30 FPS 配置启动摄像头，并在h264 视频编码后将其另存为视频文件。如果gst pipeline 状态更改为“PLAYING”，则表示摄像头正在运行。

若要停止摄像头，请按**CTRL+C**。

2. 在设备上生成/opt/mux_avc.mp4。通过在主机PC 上运行以下scp 命令，可以从设备中提取录制的内容：

```
$ scp -r root@[ip-addr]:/opt/mux_avc.mp4 .
```

视频编码和拍照

1. 在设备终端运行以下命令：

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
mux_avc.mp4 \
camsrc.image_1 ! "image/jpeg,width=1280,height=720,framerate=30/1" \
! multifilesink location=/opt/frame%d.jpg async=false sync=true
enable-last-sample=false
```

2. 按下**Enter**。此命令将打印以下菜单并等待用户输入。

```
##### MENU #####
#####

===== Pipeline
Controls=====
(0) NULL: Set the pipeline into NULL state
(1) READY: Set the pipeline into READY state
(2) PAUSED: Set the pipeline into PAUSED state
(3) PLAYING: Set the pipeline into PLAYING state

=====
Other=====
(p) Plugin Mode: Choose a plugin which to control
```

```
(q) Quit : Exit the application
```

Choose an option:

3. 使用以下菜单步骤在录制视频时拍照。

```
(1) ready -> (3) Playing -> (p)Plugin Mode : Select (8)camerasrc ->
(37) capture-image -> (1): still - Snapshot ->(1) Snapshot count (
'guint' value for arg1)
```

4. 若要停止摄像头，按**Enter**，按**b**（返回），然后按**q**（退出）。录制的视频文件和拍照图像保存在/opt/ 中。通过在主机PC 上运行以下**scp** 命令，可以从设备中提取录制的内容：

```
$ scp -r root@[ip-addr]:/opt/<file name> .
```

QCS9075

Note: 确保摄像头（MIPI 或GMSL）传感器已连接至RB8 设备。您可以在CSI 插槽上连接OV9282 MIPI 摄像头。将OX03f10 Bayer GMSL 摄像头连接到GMSL 端口0，将OX03f10 YUV GMSL 摄像头连接到端口2 和端口3。

Note: GMSL 端口0 上存在随机CSI 错误，会导致Bayer GMSL 摄像头停止流传输。此问题将在下一GA 版本中修复。

单摄像头流启动

在设备终端运行以下命令：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! 'video/x-raw,
format=NV12,\
width=1280,height=720,framerate=30/1' ! fakesink
```

以下命令在30 FPS 配置下以720p 启动摄像头。来自摄像头传感器的帧被fakesink 丢弃。如果gst pipeline 状态更改为“PLAYING”，如下所示，则表示摄像头正在运行。由于此命令会将摄像头帧dump 到fakesink，因此不会在设备上保存任何内容。

```
gbm_create_device(187): Info: backend name is: msm_drm
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Setting pipeline to PLAYING ...
```

```
New clock: GstSystemClock
```

若要停止摄像头，请按**CTRL+C**。

视频编码

1. 在设备终端运行以下命令：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,\
video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink
location=/opt/mux_avc.mp4
```

此命令以720p、30 FPS 配置启动摄像头，并在h264 视频编码后将其另存为视频文件。如果gst pipeline 状态更改为“PLAYING”，则表示摄像头正在运行。

若要停止摄像头，请按**CTRL+C**。

2. 在设备上生成/opt/mux_avc.mp4。通过在主机PC 上运行以下scp 命令，可以从设备中提取录制的内容：

```
$ scp -r root@[ip-addr]:/opt/mux_avc. mp4 .
```

视频编码和拍照

QCS9075 没有硬件JPEG 编码器。拍照用例使用在CPU 上运行的软件JPEG 编码器算法。

1. 在设备终端运行以下命令：

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc camera=0 ! video/x-raw,
format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! queue !
v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate_mode=0,video_
bitrate=6000000;" ! queue ! \
h264parse ! mp4mux ! queue ! filesink location="/opt/mux_720_avc.mp4"
camsrc.image_1 ! "image/jpeg,\
width=1280,height=720,framerate=30/1" ! multifilesink location=/opt/
frame%d.jpg async=false sync=true \
enable-last-sample=false
```

2. 按下**Enter**。此命令将打印以下菜单并等待用户输入。

```
##### MENU #####
#####
```

```

===== Pipeline
Controls=====
(4)      NULL: Set the pipeline into NULL state
(5)      READY: Set the pipeline into READY state
(6)      PAUSED: Set the pipeline into PAUSED state
(7)      PLAYING: Set the pipeline into PLAYING state
=====
Other=====
(r)      Plugin Mode: Choose a plugin which to control
(s)      Quit : Exit the application

Choose an option:

```

3. 使用以下菜单步骤在录制视频时拍照。

```

(1) ready -> (3) Playing -> (p)Plugin Mode : Select (8)camerasrc ->
(37) capture-image -> (1): still - Snapshot ->(1) Snapshot count (
'guint' value for arg1)

```

4. 若要停止摄像头，按**Enter**，按**b**（返回），然后按**q**（退出）。录制的视频文件和拍照图像保存在/opt/ 中。通过在主机PC上运行以下**scp**命令，可以从设备中提取录制的内容：

```
$ scp -r root@[ip-addr]:/opt/<file name> .
```

MIPI 多摄像头流传输

RB8 主板上四个MIPI CSI 插槽。将OV9282 摄像头连接到MIPI CSI 端口。您可以同时运行四个摄像头中的任意两个OV9282 摄像头。

在两个不同终端中用camera = 0, 1 运行以下GMSL 命令，验证MIPI 摄像头并发性。保存每个摄像头的编码文件时使用不同的文件名。

```

gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
mux_avc.mp4

```

您还可以使用离线IFE 功能同时运行所有四个MIPI 摄像头。有关四摄像头验证的信息，请参阅[支持使用离线IFE 的多摄像头](#)。

GMSL 多摄像头流传输

GMSL 端口支持连接四个GMSL 摄像头。目前在RB8 上，以下GMSL 并发已使用gst-camera-per-port-example 应用程序验证。

GMSL 摄像头并发	支持的分辨率和FPS	连接
四个3F10 YUV GMSL 摄像头	1920x1536, 30 FPS	将4 个3F10 YUV GMSL 连接到端口2
四个3F10 YUV GMSL 摄像头	1920x1536, 30 FPS	将4 个3F10 YUV GMSL 连接到端口3
八个3F10 YUV GMSL 摄像头	1920x1536, 30 FPS	将4 个3F10 YUV GMSL 连接到端口2 将4 个3F10 YUV GMSL 连接到端口3

Note: 目前，端口0 上的3F10 Bayer GMSL 摄像头存在问题，因此尚未验证。

要使用gst-camera-per-port-example 运行八个3F10 YUV GMSL 摄像头并发用例：

1. 将四个3F10 YUV GMSL 连接到端口2，将四个3F10 YUV GMSL 连接到端口3。
2. 运行gst-camera-per-port-example 应用程序。

```
# gst-camera-per-port-example
```

3. 输入您要打开的摄像头ID（空格分隔）：0 1 2 3 4 5 6 7
4. 输入八个摄像头的以下详细信息：
 - 摄像头0 的流数：1
 - 摄像头0 的流1 的宽度：1920
 - 摄像头0 的流1 的高度：1536
 - 摄像头0 的流1 的帧率：30
 - 摄像头1 的流数：1
 - 摄像头1 的流1 的宽度：1920
 - 摄像头1 的流1 的高度：1536
 - 摄像头1 的流1 的帧率：30
5. 为8 个摄像头输入GST 摄像头pipeline 命令。保存每个摄像头的编码文件时使用不同的文件名。
 - 摄像头0:

```
qtiqmmfsrc name=camsrc0 camera=0 video_0::type=video ! \
video/x-raw,format=NV12_Q08C,width=1920,height=1536,
framerate=30/1,interlace-mode=progressive,\
colorimetry=bt601 ! queue ! v4l2h264enc capture-io-mode=4
output-io-mode=5 ! h264parse ! \
mp4mux ! queue ! filesink location=/opt/cam0_IOT_1536p_NV12_
Q08C.mp4
```

- 摄像头1:

```
qtiqmmfsrc name=camsrc1 camera=1 video_0::type=video ! \
video/x-raw,format=NV12_Q08C,width=1920,height=1536,
framerate=30/1,interlace-mode=progressive,\
colorimetry=bt601 ! queue ! v4l2h264enc capture-io-mode=4
output-io-mode=5 ! h264parse ! \
mp4mux ! queue ! filesink location=/opt/cam1_IOT_1536p_NV12_
Q08C.mp4
```

终端上应显示以下内容:

```
Setting pipeline gst-camera-pipeline for camera 0 to PLAYING
Pipeline is PREROLLING ...
Setting pipeline gst-camera-pipeline for camera 1 to PLAYING
Pipeline is PREROLLING ...
Pipelines are started
```

pipeline 将从NULL 状态变为播放状态, 并且八个GMSL 摄像头都将开始mp4 dump。

要停止摄像头流传输, 请选择(q)。退出应用程序时显示以下日志:

```
Quit pressed!!

gst-camera-pipeline for camera 0 Received End-of-Stream...
Setting pipeline to NULL
Pipeline state change was successful

gst-camera-pipeline for camera 1 Received End-of-Stream...
Setting pipeline to NULL
Pipeline state change was successful
g_main_loop_run ends
```

QCS8275

Note: 确保GMSL 摄像头已连接到Ride SX 设备。您可以将OX03f10 Bayer GMSL 摄像头连接到GMSL 端口0，将OX03f10 YUV GMSL 摄像头连接到端口1 和端口2。

单摄像头流启动

在设备终端运行以下命令：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! 'video/x-raw,
format=Nv12,\
width=1280,height=720,framerate=30/1' ! fakesink
```

以下命令在30 FPS 配置下以720p 启动摄像头。来自摄像头传感器的帧被fakesink 丢弃。如果gst pipeline 状态更改为“PLAYING”，如下所示，则表示摄像头正在运行。由于此命令会将摄像头帧dump 到fakesink，因此不会在设备上保存任何内容。

```
gbm_create_device(187): Info: backend name is: msm_drm
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
```

若要停止摄像头，请按**CTRL+C**。

视频编码

1. 在设备终端运行以下命令：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=Nv12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,
video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink
location=/opt/mux_avc.mp4
```

此命令以720p、30 FPS 配置启动摄像头，并在h264 视频编码后将其另存为视频文件。如果gst pipeline 状态更改为“PLAYING”，则表示摄像头正在运行。

若要停止摄像头，请按**CTRL+C**。

2. 在设备上生成/opt/mux_avc.mp4。通过在主机PC 上运行以下scp 命令，可以从设备中提取录制的内容：

```
$ scp -r root@[ip-addr]:/opt/mux_avc. mp4 .
```

视频编码和拍照

QCS8275 没有硬件JPEG 编码器。拍照用例使用在CPU 上运行的软件JPEG 编码器算法。

1. 在设备终端运行以下命令：

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc camera=0 ! video/x-raw,
format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! queue !
v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate_mode=0,video_
bitrate=6000000;" ! queue ! \
h264parse ! mp4mux ! queue ! filesink location="/opt/mux_720_avc.mp4"
camsrc.image_1 ! "image/jpeg,\
width=1280,height=720,framerate=30/1" ! multifilesink location=/opt/
frame%d.jpg async=false sync=true \
enable-last-sample=false
```

2. 按下**Enter**。此命令将打印以下菜单并等待用户输入。

```
##### MENU #####
#####

===== Pipeline
Controls=====
(4)      NULL: Set the pipeline into NULL state
(5)      READY: Set the pipeline into READY state
(6)      PAUSED: Set the pipeline into PAUSED state
(7)      PLAYING: Set the pipeline into PLAYING state
=====
Other=====
(r)      Plugin Mode: Choose a plugin which to control
(s)      Quit : Exit the application

Choose an option:
```

3. 使用以下菜单步骤在录制视频时拍照。

```
(1) ready -> (3) Playing -> (p)Plugin Mode : Select (8)camerasrc ->
(37) capture-image -> (1): still - Snapshot ->(1) Snapshot count (
'guint' value for arg1)
```

4. 若要停止摄像头，按**Enter**，按**b**（返回），然后按**q**（退出）。录制的视频文件和拍照图像保存在/opt/ 中。通过在主机PC 上运行以下**scp** 命令，可以从设备中提取录制的内

容:

```
$ scp -r root@[ip-addr]:/opt/<file name> .
```

其他GStreamer 示例

QIM SDK 包含[GStreamer 摄像头示例应用](#)以及用于[AI/ML 和其他多媒体应用](#)的示例应用。

Note: 在使用示例应用程序之前，请确保满足gst-launch-1.0 和GStreamer 插件的安装先决条件。

有关使用gst-launch-1.0 的示例，请参阅[多媒体用例示例](#)。

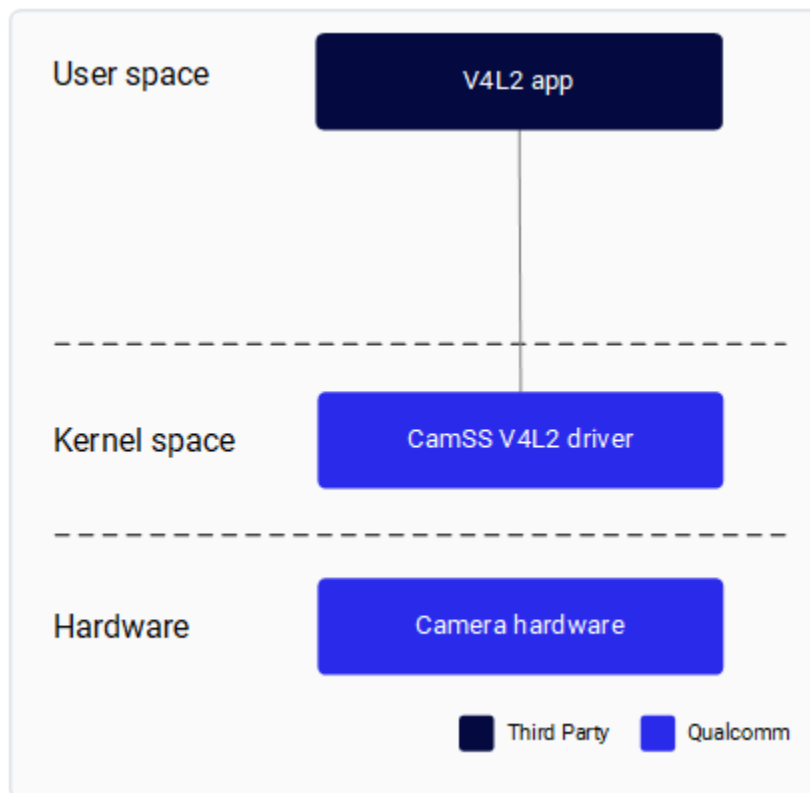
使用V4L2 API 进行摄像头流传输

QCS6490

Linux 内核中的V4L2 框架支持视频设备。它提供了一个API，允许用户空间应用程序与摄像头和视频采集卡等设备进行交互。

有关V4L2 的更多信息，请访问kernel.org。

Qualcomm 支持V4L2 接口摄像头ISP 驱动程序，以实现上游内核中的原始帧dump 功能。



CamSS 驱动程序

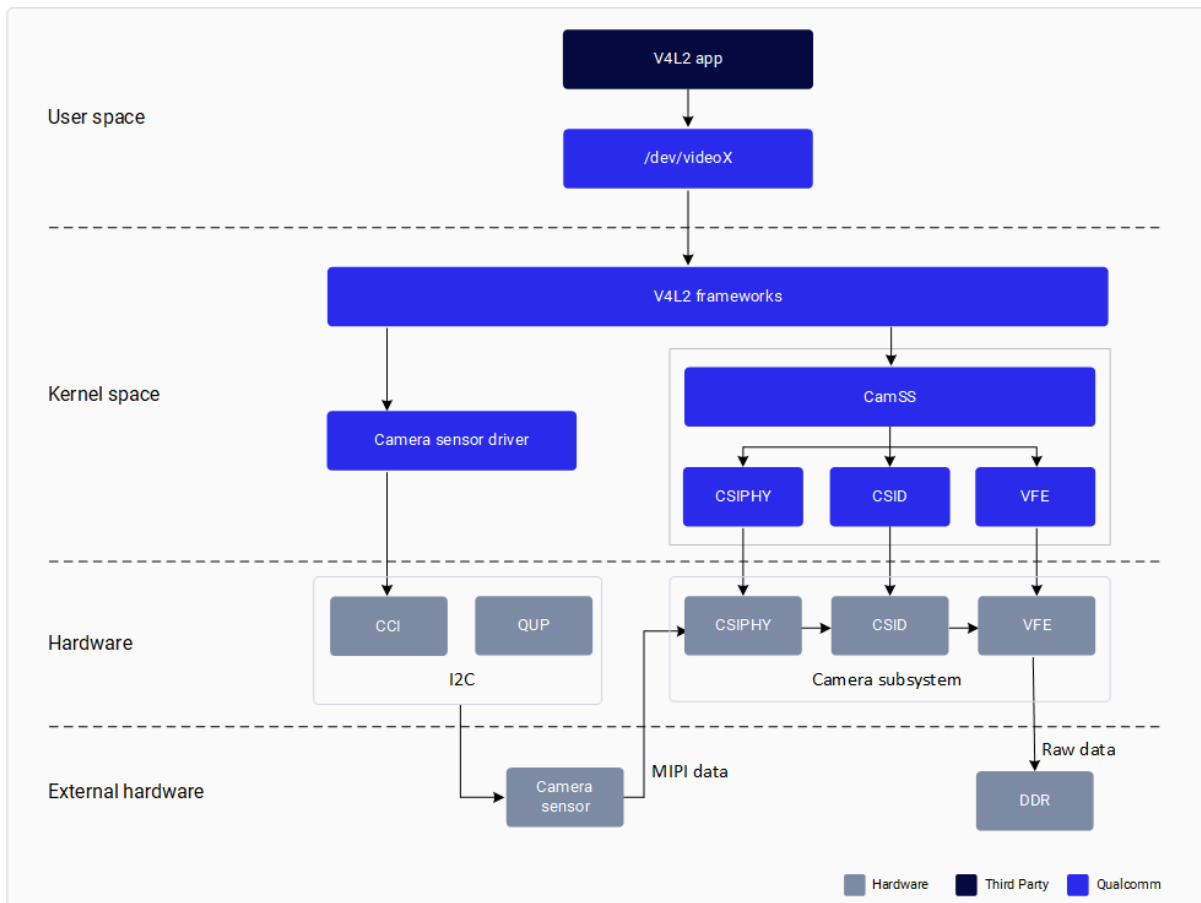
上游内核中的Qualcomm 摄像头子系统(CamSS) 驱动程序实现V4L2、媒体控制器和V4L2 subdev 接口。

支持使用内核中V4L2 子设备接口的摄像头传感器。

CamSS 驱动程序包括：

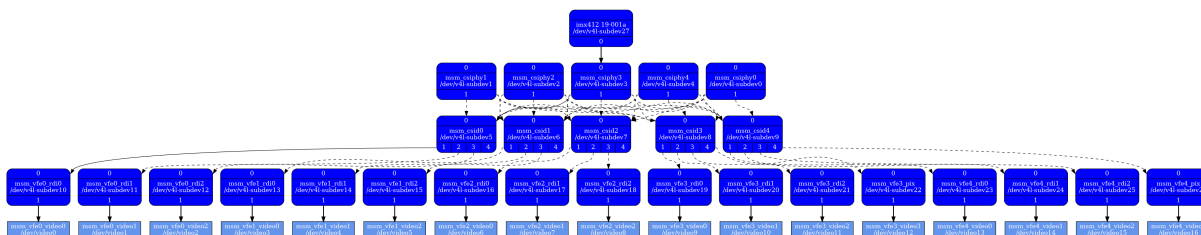
- CSIPHY 模块– 处理CSI2 接收器的物理层。每个CSIPHY 模块都可以连接一个单独的摄像头传感器。
- CSI 解码器(CSID) 模块– 处理CSI2 接收器的协议层和应用层。CSID 可以解码来自任何CSIPY 的数据流。

- 视频前端(VFE) 模块– 表示图像前端(IFE)， 包含绕过图像处理pipeline 的原始数据转储接口(RDI) 输入接口。VFE 还包含AXI 总线接口， 该接口将输出数据写入存储器。



CamSS 驱动程序实现V4L2 接口。每个CSIPHY、CSID 和VFE 模块由单个V4L2 子设备表示。

如下图所示，每个CSIPHY 可以连接到各CSID。每个CSID 可以连接到各VFE。每个RDI 端口都有一个单独的视频节点。



CamSS V4L2 驱动程序位于`<kernel_root>/drivers/media/platform/qcom/camss`。 `<kernel_root>` 是Linux 内核的目录。

以下代码片段显示了CamSS 驱动程序的v4l2_subdev_internal_ops。

```
static const struct v4l2_subdev_internal_ops csiphy_v4l2_internal_ops
= {
    .open = csiphy_init_formats,
};
...
static const struct v4l2_subdev_internal_ops csid_v4l2_internal_ops =
{
    .open = csid_init_formats,
};
...
static const struct v4l2_subdev_internal_ops vfe_v4l2_internal_ops =
{
    .open = vfe_init_formats,
};
```

CamSS 驱动程序仅使用开放接口初始化受支持的格式。当应用程序打开 **subdev** 设备节点时，将调用它。

以下代码片段显示了 **CSID** 模块的 `v4l2_subdev_ops`。 **CSIPHY** 和 **VFE** 模块具有类似的 `v4l2_subdev_ops`。

```
static const struct v4l2_subdev_ops csid_v4l2_ops = {
    .core = &csid_core_ops,
    .video = &csid_video_ops,
    .pad = &csid_pad_ops,
};
...
static const struct v4l2_subdev_core_ops csid_core_ops = {
    .s_power = csid_set_power,
    .subscribe_event = v4l2_ctrl_subdev_subscribe_event,
    .unsubscribe_event = v4l2_event_subdev_unsubscribe,
};

static const struct v4l2_subdev_video_ops csid_video_ops = {
    .s_stream = csid_set_stream,
};

static const struct v4l2_subdev_pad_ops csid_pad_ops = {
    .enum_mbus_code = csid_enum_mbus_code,
    .enum_frame_size = csid_enum_frame_size,
    .get_fmt = csid_get_format,
    .set_fmt = csid_set_format,
};
```

这些接口由 **V4L2** 框架或驱动程序在各种上下文中调用（例如 `ioctl`、`setup_link`、`start_`

streaming) 。例如:

```
v4l2_subdev_call(subdev, pad, get_fmt, NULL, &fmt);``
```

以下代码片段显示了**CamSS** 驱动程序的media_device_ops 和**CSIPHY**、**CSID** 和**VFE** 模块的media_entity_operations。

```
static const struct media_device_ops camss_media_ops = {
    .link_notify = v4l2_pipeline_link_notify,
};
...
static const struct media_entity_operations csiphy_media_ops = {
    .link_setup = csiphy_link_setup,
    .link_validate = v4l2_subdev_link_validate,
};
...
static const struct media_entity_operations csid_media_ops = {
    .link_setup = csid_link_setup,
    .link_validate = v4l2_subdev_link_validate,
};
...
static const struct media_entity_operations vfe_media_ops = {
    .link_setup = vfe_link_setup,
    .link_validate = v4l2_subdev_link_validate,
};
```

在注册**CamSS** 驱动程序时注册Camss_media_ops (camss_probe)。**CSIPHY**、**CSID** 和**VFE** 媒体实体操作。link_setup 在media_device_setup_link 的上下文中调用。

V4L2 示例应用程序- Yavta

本节介绍了如何使用支持**V4L2** 接口的应用程序采集原始帧数据。

启用**CamSS** 驱动程序

1. 使用以下命令下载上游内核源代码。

```
devtool modify linux-qcom-custom
```

这会将内核源代码下载到<WORKSPACE>/build-qcom-wayland/workspace/sources/linux-qcom-custom/。

2. 应用以下更改，以在设备树中启用**CamSS** 驱动程序:

```
<WORKSPACE>/build-qcom-wayland/workspace/sources/linux-qcom-custom/
arch/ arm64/boot/dts/qcom/qcs6490-addons-rb3gen2.dtsi

&camss {
```

```

-     status = "disabled";
+     status = "okay";
    ports {
        #address-cells = <1>;
        #size-cells = <0>;
csiphy3_ep: endpoint {
};

&ccil {
-     status = "disabled";
+     status = "okay";
};

```

3. 编辑<workspace>/layers/meta-qcom-hwe/recipes-kernel/linux/linux-qcom-custom 中的linux-qcom-custom_%.bbappend 或.bb 文件，以包含步骤3 中的补丁：

```
SRC_URI += "file://camss_enable_change.patch"
```

or

```
SRC_URI:qcom-custom-bsp += "file://camss_enable_change.patch"
```

4. 按照[编译指南](#)中的Yocto 编译说明编译镜像。
5. 按照[刷写镜像](#)中的说明刷写镜像。

编译并推送媒体控制器实用工具和Yavta 应用程序

1. 编译Yavta 和媒体控制器。

```
bitbake yavta
```

2. 将二进制文件推送到设备。在以下示例中，<workspace> 是Qualcomm 软件版本的目录。

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

Note: ipk 文件名中的版本号可能与您的编译版本不同。在您的编译中使用v4l-utils、yavta、media-ctl 和libv4l。

```
scp <WORKSPACE>/build-qcom-wayland/tmp-glibc/deploy/ipk/armv8-2a/v4l-
utils_1.22.1-r0_armv8-2a.ipk root@[ip-addr]:/var/cache/camera/
```

```
scp <WORKSPACE>/build-qcom-wayland/tmp-glibc/deploy/ipk/armv8-2a/
yavta_0.0-r2_armv8-2a.ipk root@[ip-addr]:/var/cache/camera/
scp <WORKSPACE>/build-qcom-wayland/tmp-glibc/deploy/ipk/armv8-2a/
media-ctl_1.22.1-r0_armv8-2a.ipk root@[ip-addr]:/var/cache/camera/
scp <WORKSPACE>/build-qcom-wayland/tmp-glibc/deploy/ipk/armv8-2a/
libv4l_1.22.1-r0_armv8-2a.ipk root@[ip-addr]:/var/cache/camera/
```

3. 创建与设备的shell 连接:

```
ssh root@[ip-addr]
```

4. 禁用摄像头模块。

- 摄像头模块不能与CamSS 驱动程序共存。
- 将camera.ko 模块从/lib/modules/6.6.90-qli-1.5-*/camera/ 中移出，确保其不会自动加载，然后重启设备。

```
# mount -o rw,remount /usr
# mv /lib/modules/6.6.90-qli-1.5-*/camera/camera*.ko /
# reboot
```

5. 创建与设备的shell 连接。

```
ssh root@[ip-addr]
```

6. 安装media-ctl、libv4l、v4l-utils 和yavta 软件包。

```
# mount -o rw,remount /usr

# opkg --nodeps install /var/cache/camera/media-ctl_1.22.1-r0_armv8-
2a.ipk --force-reinstall
# opkg --nodeps install /var/cache/camera/libv4l_1.22.1-r0_armv8-2a.
ipk --force-reinstall
# opkg --nodeps install /var/cache/camera/v4l-utils_1.22.1-r0_armv8-
2a.ipk --force-reinstall
# opkg --nodeps install /var/cache/camera/yavta_0.0-r2_armv8-2a.ipk -
-force-reinstall
```

7. 可选择添加传感器驱动程序和CamSS 模块。

```
# modprobe imx412
# modprobe qcom-camss
```

这是一个可选步骤，因为/lib/modules 中的imx412 和qcom-camss 模块是自动加载的。Modprobe 用于在Linux 内核中添加/删除模块。imx412 和qcom-camss 模块位于设备上的以

下路径中:

- /lib/modules/*/kernel/drivers/media/i2c/imx412.ko
- /lib/modules/*/kernel/drivers/media/platform/qcom/camss/qcom-camss.ko

可使用以下`lsmod` 命令验证`qcom_camss` 和`imx412` 模块的加载:

```
# lsmod | grep qcom_camss # lsmod | grep imx412
```

检查媒体节点编号

运行以下命令, 打印**CamSS** 驱动程序的媒体设备节点编号。

如果`/dev/media0` 未列出`qcom-camss` 驱动程序, 请尝试使用`/dev/media1`。

```
# media-ctl -p -d /dev/media0 | grep camss
driver      qcom-camss
bus info     platform:acaf000.camss
```

查找传感器名称

运行以下命令, 将传感器名称打印到终端:

```
# cat /sys/dev/char/81\:*/*name | grep imx imx412 19-001a
imx577 17-001a
```

配置媒体控制器

媒体控制器工具(`media-ctl`) 是用于配置摄像头子系统设备的**V4L2 utility**。使用`media-ctl --help` 打印使用信息。

Note: 将`[x]` 替换为通过检查媒体节点编号找到的数字。<Check the media node number>。例如, `# media-ctl -d /dev/media0 --reset`。

1. 将所有链接重置为非活动状态:

```
# media-ctl -d /dev/media[x] --reset
```

2. 在**pipeline** 节点上配置摄像头传感器格式和分辨率:

```
# media-ctl -d /dev/media[x] -V '"imx577 17-001a":0[fmt:SRGB10/4056x3040 field:none]'
```

3. 配置**CSIPHY** 的分辨率为**4056 x 3040**:


```
# media-ctl -d /dev/media[x] -V '"msm_csiphy3":0[fmt:SRGB10/4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_csiphy3":1[fmt:SRGB10/4056x3040]'
```

4. 配置CSID 的分辨率为4056 x 3040:

```
# media-ctl -d /dev/media[x] -V '"msm_csid0":0[fmt:SRGB10/4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_csid0":1[fmt:SRGB10/4056x3040]'
```

5. 配置ISP 分辨率为3840 x 2160:

```
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":0[fmt:SRGB10/4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":1[fmt:SRGB10/4056x3040]'
```

6. 链接pipeline:

```
# media-ctl -d /dev/media[x] -l '"msm_csiphy3":1->"msm_csid0":0[1]'
# media-ctl -d /dev/media[x] -l '"msm_csid0":1->"msm_vfe0_rdi0":0[1]'
```

采集图像

[Yavta test application](#) 使用V4L2 接口验证摄像头。运行Yavta 以采集图像:

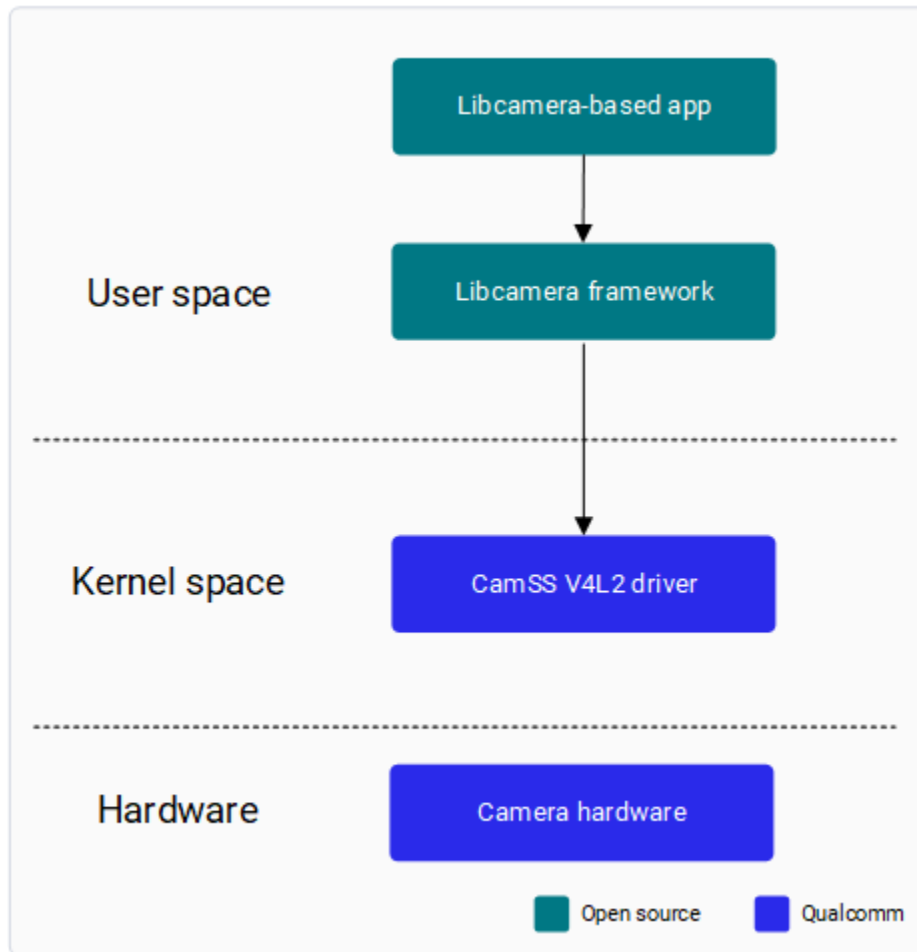
```
# yavta -B capture-mplane -c -I -n 5 -f SRGB10P -s 4056x3040 -F /dev/video0 --capture=5 --file='frame-#.raw'
```

V4L2 示例应用程序- libcamera

Note: GA1.5 版本中使用的libcamera 版本存在一个已知的功能问题，该问题将在下一个版本中修复。如果对libcamera 有任何要求，请联系Qualcomm 寻求指导。

libcamera 是一个开源软件框架。它处理对V4L2 摄像头接口的控制，并向上层提供原生C++ API。应用程序和上层框架基于[libcamera framework](#) 运行。

有关libcamera 架构的更多详细信息，请参阅[libcamera architecture](#)。



使用 `cam utility` 对 `libcamera` 进行验证。

采集原始帧数据

以下步骤介绍如何使用 `V4L2` 接口通过 `cam utility` 程序采集原始帧数据。

1. 启用 `CamSS` 驱动程序。

- a. 使用以下命令下载上游内核源代码。

```
devtool modify linux-qcom-custom
```

这会将内核源代码下载到 `<WORKSPACE>/build-qcom-wayland/workspace/sources/linux-qcom-custom/`。

- b. 应用以下更改，以在设备树中启用 `CamSS` 驱动程序：

```
<WORKSPACE>/build-qcom-wayland/workspace/sources/linux-qcom-custom/arch/arm64/boot/dts/qcom/qcs6490-addons-rb3gen2.dtsi
```

```
&camss {
-     status = "disabled";
+     status = "okay";
    ports {
        #address-cells = <1>;
        #size-cells = <0>;
csiphy3_ep: endpoint {
};

&ccil {
-     status = "disabled";
+     status = "okay";
};
```

2. 编辑<workspace>/layers/meta-qcom-hwe/recipes-kernel/linux/linux-qcom-custom 中的linux-qcom-custom_%.bbappend 或.bb 文件，以包含步骤1 中的补丁：

```
SRC_URI += "file://camss_enable_change.patch"
```

or

```
SRC_URI:qcom-custom-bsp += "file://camss_enable_change.patch"
```

3. 按照[编译指南](#)中的Yocto 编译命令编译镜像。
4. 按照[刷写镜像](#)中的说明刷写镜像。
5. 禁用摄像头模块。

摄像头模块不能与CamSS 驱动程序共存。将camera.ko 模块从/lib/modules/6.6.90-qli-1.5-*/camera/* 中移出，确保其不会自动加载，然后重启设备。

```
# mount -o rw,remount /usr
# mv /lib/modules/6.6.90-qli-1.5-*/camera/camera*.ko /
# reboot
```

6. 编译并推送libcamera 工具。

```
bitbake libcamera
```

7. 将二进制文件推送到设备。在以下示例中，<workspace> 是Qualcomm 软件版本的目录。

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

```
scp <WORKSPACE>/build-qcom-wayland/tmp-glibc/deploy/ipk/armv8-2a/
libcamera_202105+git0+acf8d028ed-r0_armv8-2a.ipk root@[ip-addr]:/var/
cache/camera/
scp <WORKSPACE>/build-qcom-wayland/tmp-glibc/deploy/ipk/armv8-2a/
libevent-pthreads-2.1-7_2.1.12-r0_armv8-2a.ipk root@[ip-addr]:/var/
cache/camera/
scp <WORKSPACE>/build-qcom-wayland/tmp-glibc/deploy/ipk/armv8-2a/
libevent-2.1-7_2.1.12-r0_armv8-2a.ipk root@[ip-addr]:/var/cache/
camera/
```

8. 创建与设备的shell 连接:

```
ssh root@[ip-addr]
```

9. 可选择添加传感器驱动程序和CamSS 模块。

```
# modprobe imx412
# modprobe qcom-camss
```

这是一个可选步骤，因为/lib/modules 中的imx412 和qcom-camss 模块是自动加载的。Modprobe 用于在Linux 内核中添加/删除模块。imx412 和qcom-camss 模块位于设备上的以下路径中：

- /lib/modules/*/kernel/drivers/media/i2c/imx412.ko
- /lib/modules/*/kernel/drivers/media/platform/qcom/camss/qcom-camss.ko

可使用以下lsmod 命令验证qcom_camss 和imx412 模块的加载：

```
# lsmod | grep qcom_camss
# lsmod | grep imx412
```

10. 安装libcamera 工具

```
# mount -o rw,remount /usr

# opkg --nodeps install /var/cache/camera/libcamera_
202105+git0+acf8d028ed-r0_armv8-2a.ipk --force-reinstall
# opkg --nodeps install /var/cache/camera/libevent-pthreads-
2.1-7_2.1.12-r0_armv8-2a.ipk --force-reinstall
# opkg --nodeps install /var/cache/camera/libevent-2.1-7_2.
1.12-r0_armv8-2a.ipk --force-reinstall
```

11. 运行以下命令删除uncalibrated.yaml:

```
# rm -rf /usr/share/libcamera/ipa/simple/uncalibrated.yaml
```

12. 运行cam utility:

```
# cam -c 1 --capture=10 --file='frame-#.raw'
```

这将运行该工具，采集根目录中的10个原始帧并将它们保存到文件中。

检查控制台日志消息，了解有关转储图像的分辨率和格式的信息。

```
Using camera /base/soc@0/cc1@ac4b000/i2c-bus@1/camera@1a as cam0 [0:
15:01.067615799] [2155] INFO Camera camera.cpp:945 configuring
streams:
(0) 4056x3040-SRGGB10_CSI2P
cam0: Capture 10 frames
```

QCS9075

Note: 本节仅适用于QCS6490。

QCS8275

Note: 本节仅适用于QCS6490。

3.4 USB 摄像头

Qualcomm Linux 设备为符合USB 视频类（UVC）标准的USB web 摄像头提供驱动程序支持。UVC 视频驱动程序将这类摄像头呈现为V4L2 视频设备，可通过字符设备节点（如/dev/videoX）进行访问。有关USB 摄像头的使用，请参阅[USB 摄像头配置](#)。示例应用程序[gst-usb-single-camera-app](#)可用于USB 摄像头验证。

3.5 网络摄像头

可使用开源GStreamer 插件支持网络摄像头。从网络获取摄像头流取决于GStreamer 插件支持的网络协议。开发者可以使用开源GStreamer 插件（如rtspsrc）处理来自网络的摄像头流，并根据需要创建pipeline。有关插件的详细信息，请参阅[rtspsrc](#)。

示例应用程序[gst-ai-multistream-inference](#) 也可用于网络摄像头验证。有关在Qualcomm 平台上开发GStreamer 应用程序的信息，请参阅[Qualcomm Intelligent Multimedia SDK](#)。

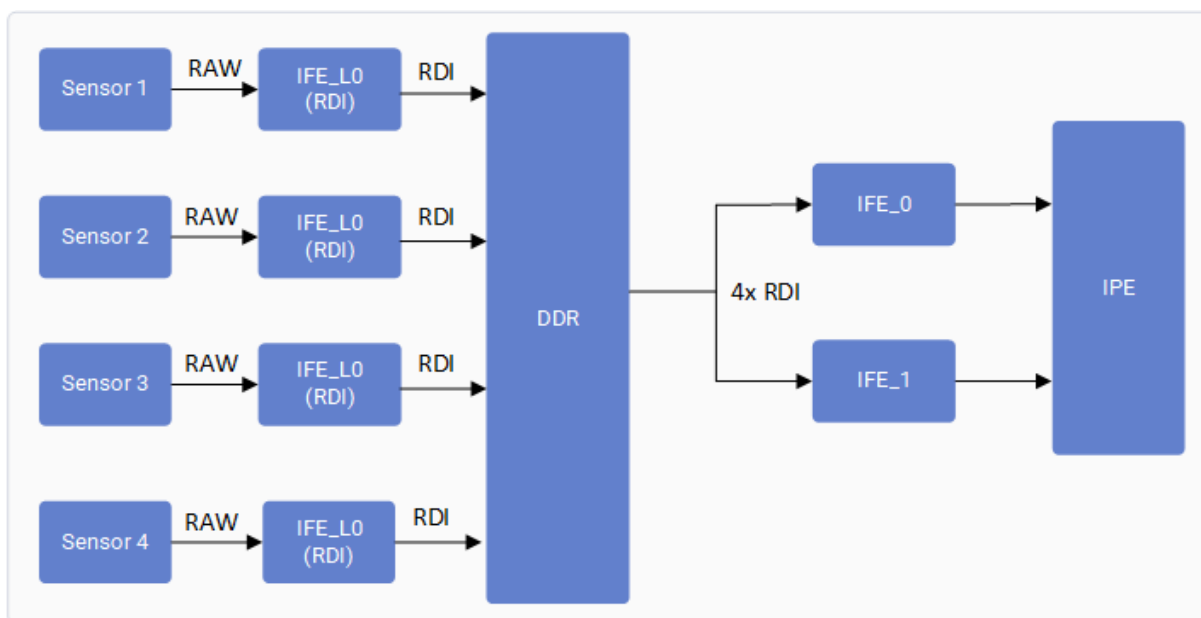
4 增强摄像头输出

4.1 支持使用离线IFE的多摄像头

Note: 本节仅适用于QCS9075。

QCS9075 有两个IFE，支持实时对Bayer 摄像头图像进行去Bayer 处理（Bayer 到YUV 处理）。这可以支持来自两个Bayer 摄像头的并发流传输。

离线IFE 功能允许IFE 在离线模式下运行，并支持使用单个IFE 对两个摄像头进行去Bayer 处理。这可以支持来自四个Bayer 摄像头的并发流传输。



每个传感器都连接到一个IFE-Lite 硬件实例，并使用RDI 端口将数据dump 到DDR。单个IFE 硬件用于通过总线提取引擎SFE Lite 从两个IFE-Lite 读取数据，并在离线模式下处理数据。

通过这种方法，摄像头帧不是直接传送到IFE，而是从IFE_LITE 路由并dump 到DDR。然后，使用提取引擎SFE Lite 将它们提供给IFE 进行Bayer 到YUV 的处理。

通过启用此功能，可以运行以下摄像头并发用例：

- 四个OV9292 MIPI 摄像头连接到4 个MIPI CSI 插槽0、1、2 和3

测试步骤

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

要收集用户日志，请在设备中运行以下命令：

```
# journalctl -f > /opt/user_log.txt
```

要收集内核日志，请在设备中运行以下命令：

```
# dmesg -w > /opt/kernel_log.txt
```

对每个摄像头运行带有multicamera-hint=true 的GST 命令以启用此功能。例如：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 multicamera-  
hint=true ! \  
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\  
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \  
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_  
bitrate=6000000,\  
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/  
mux_avc.mp4
```

同样，使用multicamera-hint=true 选项运行其他摄像头。

日志验证

检查用户日志中的以下打印：

```
CamX: [CORE_CFG]3509 3556 [CORE    ] camxpipeline.h:4222
SetPipelineStatus() RealTimeFeatureZSLPreviewRawOfflineIFE_0_cam_0
status is now PipelineStatus::STREAM_ON
```

检查内核日志中的以下打印：

```
CAM_INFO: CAM-ISP: cam_ife_hw_mgr_print_acquire_info: 1733: 0:4:11.
835 Acquired Single IFE[0] SFE[0] OFFLINE: Y with [9 pix] [0 pd] [0
rdi] ports for ctx:1
```

4.2 支持软件TNR/MCTF

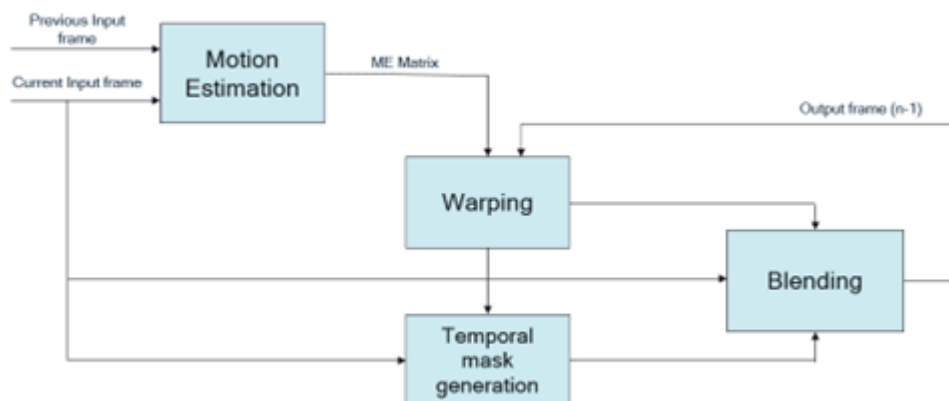
Note: 本节适用于QCS9075 和QCS8275。

运动补偿时域滤波器(MCTF) 是一种通过降低噪声并增强运动清晰度来提升预览/视频质量的技术。

在QCS9075 和QCS8275 中，IPE 硬件不支持MCTF。因此，采用了基于软件的解决方案，并在GPU 上运行。

SWMCTF 在静态区域执行帧间混合以减少噪声。此外，SWMCTF 是对全局运动补偿变换的改进。

下图展示了SWMCTF 的工作流。



MCTF 的第一步是确定当前帧和前一帧之间的运动估计矩阵。运动估计矩阵和测得的变换置信度被送入变形和混合处理。

接下来是变形。变形是基于ME 矩阵在前一输出帧上完成的。还会在当前帧和前一帧之间计算alpha 遮罩（用于对背景和局部运动进行分类）。最后，将当前输入帧和之前的输出混合，使用alpha 遮罩值生成当前输出帧。同时还考虑了为每帧计算的噪声。

使用以下GST 命令测试软件TNR:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc video_0::type=preview sw-
tnr=true ! \
video/x-raw,format=Nv12_Q08C,width=1280,height=720,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;"
! \
h264parse ! mp4mux ! filesink location=/opt/cam_prev.mp4
```

日志验证:

```
CamX: [CORE_CFG]1332 3839 [CORE   ] camxpipeline.h:4222
SetPipelineStatus() RealTimeFeatureZSLPreviewRawSWMCTF_0_cam_0 status
is now PipelineStatus::STREAM_ON
libmctf: mctf_init: 107:MCTF Version 3.14
```

4.3 启用高动态范围

Note: 本节仅适用于QCS6490。

交错式高动态范围(SHDR) 模式是一种传感器功能，可输出具有不同曝光时间的帧，即长曝光帧(LEF) 和短曝光帧(SEF)。

- 传感器将两条线作为一个单元输出。LEF 的帧和SEF 的帧在这两条线中交替输出。
- 卷帘快门读出是交错的（行交错），因此在对长曝光进行采样后立即开始短曝光（在同一帧内）。这一模式也称为数字重叠(DOL) 模式。
- SHDR 传感器能够使用单个虚拟通道（单个帧交错包含LEF 和SEF）或使用不同的虚拟通道（LEF 和SEF 帧分别在不同的虚拟通道中输出）输出LEF 和SEF 帧。
- Qualcomm 芯片组有两种解决方案（SHDR v3 和SHDR v2）。SHDR v3 适用于双虚拟通道。SHDR v2 适用于单虚拟通道。

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

请在设备上运行以下命令，以收集日志:

```
# journalctl -f > /opt/log.txt
```

SHDR v3 – 具有双帧曝光的双虚拟通道

SHDR v3 解决方案使用传感器，可以在不同的CSI 虚拟通道上发送曝光帧（LEF 和SEF）。长曝光帧和短曝光帧在不同的虚拟通道上传输。

使用以下GStreamer 命令启用SHDR v3:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 vhdr=2 ! \
video/x-raw,format=Nv12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_4k.mp4
```

视频文件保存在/opt/cam_4k.mp4。

请检查以下日志，以便验证是否选择了SHDR v3 用例:

```
cam-server: CamX: [CORE_CFG]891 23288 [CORE ] camxpipeline.h:3024
SetPipelineStatus() RealTimeYUVSHDR_0 status is now PipelineStatus::
STREAM_ON
```

SHDR v2 – 具有两帧曝光的单虚拟通道

SHDR v2 解决方案要求传感器以交错方式（也称为DOL 模式）在单个虚拟通道上同时输出LEF 和SEF。卷帘快门读出是交错的（行交错），因此在对长曝光进行采样后立即开始短曝光（在同一帧内）。

使用以下GStreamer 命令启用SHDR v2

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 vhdr=1 ! \
video/x-raw,format=Nv12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_4k.mp4
```

视频文件保存在/opt/cam_4k.mp4。

要验证是否选择了SHDR v2 用例，请检查以下日志:

```
cam-server: CamX: [CORE_CFG]852 2070 [CORE] camxpipeline.h:3015
SetPipelineStatus()
RealTimeSHDR_0 status is now PipelineStatus::STREAM_ON
```

除雾功能

除雾是一种雾检测和清除技术。该功能允许用户在下雨、烟雾、雾霾或浓雾等恶劣天气状况下消除雾效应。它通过提高SHDR v2 和SHDR v3 用例中的图像质量来提供除雾图像。除雾库使用从IFE、BPS 和IPE 收集的统计信息和插值数据进行除雾操作，并为IQ 模块生成新表以便在下一帧中应用。

要验证除雾功能，请执行以下操作：

1. 使用/var/cache/camera/camoverridesettings.txt 中的shdrModeType=5 覆盖设置可在SHDR 用例中启用除雾功能。
2. 使用/var/cache/camera/camoverridesettings.txt 中的下列覆盖设置，验证除雾功能是否正常运行：

```
logInfoMask=0x40000
logVerboseMask=0x40000
```

3. 测试除雾功能。

- 运行以下命令，使用SHDR v2 用例测试除雾：

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc vhdr=1 ! \
video/x-raw,format=NV12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc
capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,video_bitrate_mode=0;" ! queue ! \
h264parse ! mp4mux ! queue ! filesink location=/opt/cam_4k.mp4
```

- 运行以下命令，使用SHDR v3 用例测试除雾：

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc vhdr=2 ! \
video/x-raw,format=NV12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc
capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
video_bitrate_mode=0;" ! queue ! \
h264parse ! mp4mux ! queue ! filesink location=/opt/cam_4k.mp4
```

4. 启用除雾。

- a. 选择(1) READY。

- b. 选择**(3) PLAYING**。
- c. 按下**Enter**。
- d. 选择**(p) Plugin Mode**。
- e. 选择**(8) camsrc**。
- f. 选择**(24) defog-table**。
- g. 输入以下字符串以启用除雾。

```
org.quic.camera.defog, enable=true, strength=3, ates_
strength=1;
```

- h. 选择**(b) BACK**，然后选择**(q) QUIT**，关闭摄像头。视频文件保存在/opt/cam_4k.mp4。

5. 检查以下日志以验证除雾功能是否启用：

```
defog.cpp:5782 DefogInitialize() ===== Defog lib
v2.5.0=====
cam-server: CamX: [ VERB]793766 793830 [CHI ] camxchinodedefog.
cpp:1981 SetDeFog2ConfigParams() Previous ( defog_en = 0 defog_
strength = 0 ates_strength = 0
cam-server: CamX: [ VERB]793766 793830 [CHI ] camxchinodedefog.
cpp:1984 SetDeFog2ConfigParams() Current ( defog_en = 1 defog_
strength = 3 ates_strength = 1
```

4.4 启用EIS 和LDC

Note: 本节仅适用于QCS6490。

本节介绍EIS 和LDC 功能以及如何运行其各种用例。

电子图像稳定(EIS) 是一种使用电子处理的图像增强技术。EIS 可最大限度地减少模糊并补偿设备抖动。EIS 从IMU 传感器获取运动数据，并生成变换矩阵，以补偿所有三个方向上的设备运动。

陀螺仪传感器提供俯仰、偏航和侧倾运动。在GPU 上运行的图像变形库模块根据生成的变换矩阵对图像进行变形。iWarp 库使用OpenGL API 进行变形。镜头畸变校正(LDC) 是一个校正因鱼眼镜头而引起的畸变的过程，它使场景中的直线由于镜头的畸变而被拍摄为曲线。使用镜头的校准过程生成静态网格。

在GPU 上运行的图像变形库模块根据生成的静态网格对图像进行变形。iWarp 库使用OpenGL API 进行变形。

EIS 和LDC 可以独立运行，也可以同时运行。以下是EIS 和LDC 用例中可能出现的GST 命令选项：

GST 命令选项	说明
eis=1	仅在第一个流上启用EIS
eis=2	在两个流上启用EIS
ldc=1	在所有流（1 或2）上启用LDC
eis=1,ldc=1	仅在第一个流上启用EIS 和LDC
eis=2,ldc=1	在两个流上启用EIS 和LDC

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

请在设备上运行以下命令，以收集日志：

```
# journalctl -f > /opt/log.txt
```

EIS 单流用例

使用以下GStreamer 命令启用此用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls=
"controls,video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_1080p.mp4
```

使用以下日志验证是否选择了此用例：

```
cam-server[926]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
single stream is ON..
cam-server[908]: CamX: [ INFO]908 981 [CHI   ] camxchinodeeisdgv26.
cpp:1346 Initialize() m_nodeCaps 131072
cam-server[908]: CamX: [CORE_CFG]908 1796 [CORE       ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0
status is now PipelineStatus::STREAM_ON
```

在两个流中的第一个启用**EIS** 的用例

使用以下**GStreamer** 命令启用此用例:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 video_0::type=preview
! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;"
! \
h264parse ! mp4mux ! filesink location=/opt/cam_prev.mp4 camsrc. ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc capture-
io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
video_bitrate_mode=0;" \
! h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

使用以下日志验证是否选择了此用例:

```
cam-server[926]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
single stream is ON..
cam-server[926]: CamX: [ INFO]926 988 [CHI    ] camxchinodeeisdgv26.
cpp:1346 Initialize() m_nodeCaps 131072
cam-server[926]: CamX: [CORE_CFG]926 2529 [CORE      ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0
status is now PipelineStatus::STREAM_ON
```

在两个流上启用**EIS**

使用以下**GStreamer** 命令启用此用例:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=2 video_0::type=preview
! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;"
! \
h264parse ! mp4mux ! filesink location=/opt/cam_prev.mp4 camsrc. ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc capture-
io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
video_bitrate_mode=0;" \
```

```
! h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

使用以下日志验证是否选择了此用例：

```
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
dual stream is ON..
cam-server[914]: CamX: [ INFO]914 960 [CHI    ] camxchinodeeisdgv26.
cpp:1346 Initialize() m_nodeCaps 131072
cam-server[914]: CamX: [CORE_CFG]914 2421 [CORE      ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0
status is now PipelineStatus::STREAM_ON
```

LDC 单流用例

使用以下GStreamer 命令启用此用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc ldc=1 ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls=
"controls,video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_1080p.mp4
```

使用以下日志验证是否选择了此用例：

```
cam-server[926]: [INFO]: RecorderCameraContext : OpenCamera: EIS is
disabled cam-server[926]: [INFO]: RecorderCameraContext : OpenCamera:
LDC is ON..
cam-server[926]: CamX: [ INFO]926 990 [CHI    ] camxchinodeeisdgv26.
cpp:1346 Initialize() m_nodeCaps 524288
cam-server[926]: CamX: [CORE_CFG]926 2132 [CORE      ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0
status is now PipelineStatus::STREAM_ON
```

在两个流上启用LDC 的用例

使用以下GStreamer 命令启用此用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc ldc=1 video_0::type=preview
! \
video/x-raw,format= NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls=
"controls,video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
```



```
cam_prev.mp4 \
camsrc. ! video/x-raw,format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 !
v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
video_bitrate_mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

使用以下日志验证是否选择了此用例:

```
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: EIS is
disabled cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera:
LDC is ON..
cam-server[914]: CamX: [ INFO]914 966 [CHI    ] camxchinodeeisdgv26.
cpp:1346 Initialize() m_nodeCaps 524288
cam-server[914]: CamX: [CORE_CFG]914 1591 [CORE      ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0
status is now PipelineStatus::STREAM_ON
```

EIS 和LDC 单流用例

使用以下GStreamer 命令启用此用例:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 ldc=1 ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 \
! v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls=
"controls,video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_1080p.mp4
```

使用以下日志验证是否选择了此用例:

```
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
single stream is ON..
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: LDC is
ON.. cam-server[914]: CamX: [CORE_CFG]914 2621 [CORE      ]
camxpipeline.h:3024 SetPipelineStatus()
RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now PipelineStatus:
:STREAM_ON
```

在两个流中的第一个流启用**EIS** 和**LDC** 的用例

使用以下**GStreamer** 命令启用此用例:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 ldc=1 video_0::
type=preview ! \
video/x-raw,format= NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls=
"controls,video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_prev.mp4 camsrc. ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc capture-
io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
video_bitrate_mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

使用以下日志验证是否选择了此用例:

```
cam-server[2661]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
single stream is ON..
cam-server[2661]: CamX: [ INFO]2661 2663 [CHI          ]
camxchinodeeisdgvt26.cpp:1346 Initialize() m_nodeCaps 262144
cam-server[2661]: CamX: [CORE_CFG]2661 2834 [CORE ] camxpipeline.h:
3024 SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0
status is now PipelineStatus::STREAM_ON
```

在两个流上启用**EIS** 和**LDC** 的用例

使用以下**GStreamer** 命令启用此用例:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=2 ldc=1 video_0::
type=preview ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls=
"controls,video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_prev.mp4 camsrc. ! \
video/x-raw,format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 !
v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
video_bitrate_mode=0;" ! \
```

```
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

使用以下日志验证是否选择了此用例：

```
cam-server[882]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
dual stream is ON..
cam-server[882]: CamX: [ INFO]882 977 [CHI    ] camxchinodeeisdgv26.
cpp:1346 Initialize() m_nodeCaps 262144
cam-server[882]: CamX: [CORE_CFG]882 1613 [CORE      ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0
status is now PipelineStatus::STREAM_ON
```

4.5 高级功能并发

Note: 本节仅适用于QCS6490。

本节介绍运行SHDR、EIS 和LDC 的可能并发用例。

以下是可用于并发运行SHDR、EIS 和LDC 的组合：

GST 命令选项	说明
vhdr=2 eis=1	SHDR v3 应用于所有流。EIS 应用于第一个流
vhdr=2 eis=2	SHDR v3 和EIS 应用于两个流
vhdr=2 ldc=1	SHDR v3 + LDC 应用于所有流（1 或2）
vhdr=1 ldc=1	SHDR v2+ LDC 应用于所有流（1 或2）
eis=1 vhdr=2 ldc=1	SHDRV3 应用于两个流。EIS + LDC 应用于第一个流
eis=2 vhdr=2 ldc=1	SHDRV3 + EIS + LDC 应用于两个流

Note: 当前版本在SHDR v3 和EIS 并发用例方面存在稳定性问题。按下Ctrl+C 后，GST 命令不会终止。下次运行摄像头时需要重新启动设备。

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

请在设备上运行以下命令，以收集日志：

```
# journalctl -f > /opt/log.txt
```

SHDR v3 和EIS 并发用例

SHDR 和EIS 并发仅适用于SHDR v3。此功能使用SHDR v3 传感器模式，先进行SHDR v3 处理，然后进行EIS 处理。

使用以下GStreamer 命令启用SHDR v3+EIS 单流用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 vhdr=2 ! video/x-raw,
format=NV12,\
width=1920,height=1080,framerate=30/1 ! v4l2h264enc capture-io-mode=4
output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;"
! h264parse ! \
mp4mux ! filesink location=/opt/cam_1080p.mp4
```

使用以下日志验证SHDR v3 + EIS 单流用例：

```
cam-server[882]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
single stream is ON..
cam-server[882]: CamX: [ INFO]882 975 [CHI   ] camxchinodeeisdgv26.
cpp:1346 Initialize() m_nodeCaps 131072
cam-server[882]: CamX: [CORE_CFG]882 1877 [CORE       ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now
PipelineStatus::STREAM_ON
```

使用以下GStreamer 命令启用SHDR v3 + EIS 双流用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=2 vhdr=2 video_0::
type=preview ! \
video/x-raw,format= NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls=
"controls,video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_prev.mp4 \
camsrc. ! video/x-raw,format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 !
v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
video_bitrate_mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

使用以下日志验证SHDR v3 + EIS 双流用例：

```
cam-server[882]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
dual stream is ON..
cam-server[882]: CamX: [ INFO]882 952 [CHI   ] camxchinodeeisdgv26.
```

```
cpp:1346 Initialize() m_nodeCaps 131072
cam-server[882]: CamX: [CORE_CFG]882 2058 [CORE      ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now
PipelineStatus::STREAM_ON
```

SHDR v3 和LDC 并发用例

此功能使用SHDR v3 传感器模式并进行SHDR v3 处理，然后进行LDC 处理。

使用以下GStreamer 命令启用此用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc vhdr=2 ldc=1 ! video/x-raw,\
format=NV12,width=1920,height=1080,framerate=30/1 ! v4l2h264enc
capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;"
! h264parse ! mp4mux ! \
filesink location=/opt/cam_1080p.mp4
```

使用以下日志验证是否选择了此用例：

```
cam-server[2089]: [INFO]: RecorderCameraContext : OpenCamera: EIS is
disabled cam-server[2089]: CamX: [ INFO]2089 2090 [CHI  ]
camxchinodeeisdgv26.cpp:1346 Initialize() m_nodeCaps 524288
cam-server[2089]: CamX: [CORE_CFG]2089 2324 [CORE      ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now
PipelineStatus::STREAM_ON
```

SHDR v2 和LDC 并发用例

此功能使用SHDR v2 传感器模式，先进行SHDR v2 处理，然后进行LDC 处理。

使用以下GStreamer 命令启用此用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc vhdr=1 ldc=1 ! video/x-raw,\
format=NV12,width=1920,height=1080,framerate=30/1 ! v4l2h264enc
capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;"
! h264parse ! mp4mux ! \
filesink location=/opt/cam_1080p.mp4
```

使用以下日志验证是否选择了此用例：

```
cam-server[2358]: [INFO]: RecorderCameraContext : OpenCamera: EIS is
disabled cam-server[2358]: CamX: [ INFO]2358 2366 [CHI  ]
```

```
camxchinodeeisdg26.cpp:1346 Initialize() m_nodeCaps 524288
cam-server[2358]: CamX: [CORE_CFG]2358 2527 [CORE    ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeSHDR_IOTLDC_0 status is now
PipelineStatus::STREAM_ON
```

SHDR v3、EIS 和LDC 并发用例

SHDR、EIS 和LDC 并发在SHDR v3 中启用。此功能使用SHDR v3 传感器模式，先进行SHDR v3 处理，然后进行EIS 和LDC 处理。

使用以下GStreamer 命令启用SHDR v3、EIS 和LDC 并发单流用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 vhdr=2 ldc=1 ! video/
x-raw, \
format=NV12,width=1920,height=1080,framerate=30/1 ! v4l2h264enc
capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;"
! h264parse ! mp4mux ! \
filesink location=/opt/cam_1080p.mp4
```

使用以下日志验证SHDR v3、EIS 和LDC 并发单流用例：

```
cam-server[2555]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
single stream is ON..
cam-server[2555]: CamX: [ INFO]2555 2560 [CHI        ]
camxchinodeeisdg26.cpp:1346 Initialize() m_nodeCaps 262144
cam-server[2555]: CamX: [CORE_CFG]2555 2764 [CORE    ] camxpipeline.
h:3024
SetPipelineStatus() RealTimeYUVSHDREIS_IOT_0 status is now
PipelineStatus::STREAM_ON
```

使用以下GStreamer 命令启用SHDR v3、EIS 和LDC 并发双流用例：

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=2 vhdr=2 ldc=1 video_0::
type=preview ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls=
"controls,video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
cam_prev.mp4 \
camsrc. ! video/x-raw,format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 !
v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
```

```
video_bitrate_mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

使用以下日志验证SHDR v3、EIS 和LDC 并发双流用例:

```
cam-server[907]: [INFO]: RecorderCameraContext : OpenCamera: EIS on
dual stream is ON..
cam-server[907]: CamX: [ INFO]907 984 [CHI    ] camxchinodeeisdgv26.
cpp:1346 Initialize() m_nodeCaps 262144
cam-server[907]: CamX: [CORE_CFG]907 1586 [CORE      ] camxpipeline.
h:3024 SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.6 启用多个ROI 流

Note: 本节仅适用于QCS6490。

这是一项高级摄像头控制功能，允许用户从单个摄像头获得多个ROI 流，每个流显示完整FOV 中不同的ROI。其最多支持三个裁剪的ROI 流和一个完整的ROI 流。

在三个ROI 情况下，程序会发送五个要配置的流，包括一个完整的FOV 输出流、一个输入流和三个ROI 输出流。实时Pipeline 生成完整的FOV 图像。程序接收完整的FOV 图像，并将完整的FOV 缓冲区作为输入图像提交回摄像头再处理Pipeline。再处理Pipeline 从完整的FOV 图像中生成最多三个ROI 图像。

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

请在设备上运行以下命令，以收集日志:

```
# journalctl -f > /opt/log.txt
```

要验证多重ROI 功能，请执行以下操作:

1. 将HDMI 显示器连接到设备。
2. 从SSH 终端运行以下命令以设置显示器变量:

```
# export XDG_RUNTIME_DIR=/dev/socket/weston
# export WAYLAND_DISPLAY=wayland-1
```

3. 运行以下GStreamer 命令:

```
gst-pipeline-app qtivcomposer name=mixer sink_0::position="<0,
0>" sink_0::dimensions="<960, 540>" \
sink_1::position="<960, 0>" sink_1::dimensions="<960, 540>" \
sink_2::position="<0, 540>" sink_2::dimensions="<960, 540>" \
sink_3::position="<960, 540>" sink_3::dimensions="<960, 540>"
mixer. ! queue ! \
waylandsink qos=false sync=false fullscreen=true \
qtiqmmfsrc input-roi-enable=true video_0::type=preview video_1::
type=preview \
video_2::type=preview video_3::type=preview video_0::reprocess-
enable=true \
name=camsrc ! video/x-row,format=NV12,width=1920,height=1080,
framerate=30/1 ! queue ! mixer. \
camsrc. ! video/x-row,format=NV12,width=1920,height=1080,
framerate=30/1 ! queue ! mixer. \
camsrc. ! video/x-row,format=NV12,width=1920,height=1080,
framerate=30/1 ! queue ! mixer. \
camsrc. ! video/x-row,format=NV12,width=1920,height=1080,
framerate=30/1 ! queue ! mixer.
```

4. 选择**(1) READY**。
5. 选择**(3) PLAYING**。按下**Enter**。
6. 选择**(p) Plugin Mode**。
7. 选择**(12) camsrc**。您应该看到四个具有完整FOV 的预览流。
8. 选择**(32) input-roi-info**。
9. 输入以下内容:

```
<1920, 0, 1920, 1080, 0, 1080, 1920, 1080, 0, 0, 1920, 1080>
```

您应该在HDMI 上看到一个完整的FOV 和三个ROI 流。

要关闭摄像头，选择**(b) BACK**，然后选择**(q) QUIT**。

检查以下UMD 日志，以确认是否选择了多个ROI pipeline:

```
cam-server[2218]: CamX: [CORE_CFG]2218 2368 [CORE ] camxpipeline.h:
3024 SetPipelineStatus() ZSLReprocessYuvToYuvMultiRoIStreams_0 status
is now PipelineStatus::STREAM_ON
```

Note: Full FOV 流+ ROI 流的总带宽需要小于4K，即QCS6490 芯片组的最大能力。

4.7 自动切换Linear 与SHDR 模式

Note: 本节仅适用于QCS6490。

此功能支持在无需停止和启动摄像头会话的情况下，进行Linear 和SHDR v2 模式pipeline 之间的动态切换。

此功能有助于防止在需要基于Lux 值切换SHDR 与Linear 模式时中断视频会话。其还有助于减少pipeline 切换延迟，因为Pipeline（Linear 和SHDR）在会话创建期间已预先初始化。在模式切换期间，将释放pipeline 的硬件资源（传感器、IFE、IPE），然后获取并重新配置新pipeline。

Note: 此功能仅适用于SHDR v2 和Linear 模式的Pipeline。未来的版本将可用于SHDR v3 和Linear 模式。

Note: 使用SSH 连接到设备控制台。相关说明，请参见[如何使用SSH](#)。

请在设备上运行以下命令，以收集日志：

```
# journalctl -f > /opt/log.txt
```

在GST 命令中使用vhdr=3 可启用此功能。要启用该功能并在Linear 和SHDR 模式之间切换，请执行以下操作：

1. 运行gst-camera-metadata-example 应用程序，该应用程序可以支持设置供应商标记以在runtime 切换用例。例如：

```
gst-camera-metadata-example -p "qtiqmmfsrc name=camsrc camera=0
vhdr=3 ! \
video/x-raw,format=Nv12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! queue !
v4l2h264enc capture-io-mode=4 \
output-io-mode=5 ! queue ! h264parse ! mp4mux ! queue ! filesink
location=/opt/mux4k.mp4"
```

2. 在应用程序中，选择**(1) READY**。
3. 选择**(3) PLAYING**。摄像头以Linear 模式启动。
4. 选择**(4) META**。显示以下菜单：

```
-----MENU-----
(1)      video-metadata
(2)      image-metadata
```

```
(3)      static-metadata
(4)      session-metadata
```

5. 选择**(1) video-metadata**。显示以下菜单：

```
-----video-metadata-----
(1)      List all available tags
(2)      Dump all tags values in a file
(3)      Dump custom tags values in a file
(4)      Get a tag
(5)      Set a tag
```

6. 选择**(5) Set a tag**。

7. 此时将显示一个提示，提示用户输入以空格分隔的节名和标签名，不带引号。输入 `org.quic.camera.videoHDRmode modeType`。

8. 此时将显示设置该值的提示。输入**1** 切换到**SHDR** 模式。

要在**Linear** 和**SHDR** 模式之间切换，则重复此过程并根据需要输入**1**（**SHDR** 模式）和**0**（**Linear** 模式）。

要关闭摄像头，请选择**(q) QUIT** 或按**CTRL + C**。

使用以下日志验证此功能：

- **Linear** 模式

```
cam-server[1693]: CamX: [REQMAP]1693 2652 [CORE   ] camxsession.
cpp:4811
ProcessRequest() chiFrameNum: 0 <==> requestId: 1 <==>
sequenceId: 0
<==> CSLSyncId: 1 -- RealTimeFeatureZSLPreviewRawYUV_0
```

- **SHDR** 模式:

```
cam-server[1693]: CamX: [REQMAP]1693 2652 [CORE   ] camxsession.
cpp:4811
ProcessRequest() chiFrameNum: 296 <==> requestId: 297 <==>
sequenceId:
296 <==> CSLSyncId: 297 -- RealTimeSHDR_0
```

5 摄像头故障排查

如果摄像头应用程序不工作，请检查以下内容：

1. 检查摄像头模块连接和DIP 开关设置。请参阅[入门指南](#)。
2. 重新启动cam-server:

```
# systemctl restart cam-server

or:

# pkill cam-server
```

3. 运行单流视频录制用例:

```
# mount -o rw,remount /usr

gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,\
video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink
location=/opt/mux_avc.mp4
```

4. 检查传感器探头。

- a. 使用以下命令收集日志:

```
# journalctl -f > /opt/log.txt
```

- b. 在日志中搜索“probe success”。Probe success 意味着摄像头模块已通电并响应I2C 控制。如果特定传感器没有“probe success”日志，则可能是柔性线连接或摄像头模块的问题。

以下日志指示探测到一个IMX577 (0x34)、一个OV9282 (0xc0) 和两个GMSL 解串器(0x90):

```

CAM_INFO: CAM-SENSOR:cam_sensor_driver_cmd: 938: Probe
success,slot:0,slave_addr: 0x34,sensor_id:0x577, is always
on:0
CAM_INFO: CAM-SENSOR:cam_sensor_driver_cmd: 938: Probe
success,slot:1,slave_addr: 0xc0,sensor_id:0x9281, is always
on:0
CAM_INFO: CAM-SENSOR:cam_sensor_driver_cmd: 938: Probe
success,slot:4,slave_addr: 0x90,sensor_id:0x94, is always on:
0
CAM_INFO: CAM-SENSOR:cam_sensor_driver_cmd: 938: Probe
success,slot:5,slave_addr: 0x90,sensor_id:0x94, is always on:
0

```

5. 检查摄像头传感器驱动程序命令。

收集日志并搜索cam_sensor_driver_cmd。CAM_START_DEV Success 表示摄像头传感器流传输开始。CAM_STOP_DEV Success 表示摄像头传感器流传输停止。例如：

```

CAM_INFO: CAM-SENSOR:cam_sensor_driver_cmd: 1017: CAM_ACQUIRE_
DEV Success, sensor_id: 0x577,sensor_slave_addr:0x34
CAM_INFO: CAM-SENSOR:cam_sensor_driver_cmd: 1128: CAM_START_DEV
Success, sensor_id: 0x577,sensor_slave_addr:0x34
CAM_INFO: CAM-SENSOR:cam_sensor_driver_cmd: 1156: CAM_STOP_DEV
Success, sensor_id:0x577,sensor_slave_addr:0x34
CAM_INFO: CAM-SENSOR:cam_sensor_driver_cmd: 1070: CAM_RELEASE_
DEV Success, sensor_id: 0x577,sensor_slave_addr:0x34

```

6. 检查传感器流传输。

a. 启用CSID SOF/EOF IRQ 日志：

```

# mount -o rw,remount /usr
# mount -t debugfs none /sys/kernel/debug/
# echo 3 >/sys/kernel/debug/camera/ife/ife_csid_debug

```

b. 开始采集dmesg 日志：

```
#dmesg -w > /opt/dmesg.txt
```

c. 使用GST 命令启动摄像头。

d. 抓取到的日志有助于提供有关SOF 和EOF 的详细信息，并指示哪个传感器正在流传输。以下是示例日志：

```

<idle>-0          [000] d.h1. 19287.546764: cam_log_debug:
CAM_DBG: CAM-ISP: cam_ife_csid_irq: 4996: irq_status_ipp =

```

```
0x1110 cam-server-25604      [000] dNH.. 19287.561705: cam_  
log_debug:  
CAM_DBG: CAM-ISP: cam_ife_csid_irq: 4996: irq_status_ipp =  
0xee8
```

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“Qualcomm Technologies”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “Qualcomm Internal Use Only”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

THE DOCUMENTATION ACCOMPANYING THE MATERIALS AND/OR RELEVANT PRODUCTS AND SERVICE OFFERINGS MAY INCLUDE IMPORTANT USE LIMITATIONS. ANY DEVIATIONS FROM APPLICABLE USE LIMITATIONS MAY ADVERSELY IMPACT PERFORMANCE, DURABILITY, QUALITY OR SAFETY. YOU ASSUME ALL RISKS AND LIABILITIES ASSOCIATED WITH ANY DEVIATIONS.